

Machine-Checked // SAFETY: Proofs for dotnet-rs

A Feasibility Study of a Dependently-Typed Proof DSL for unsafe Rust in an ECMA-335 Virtual Execution System

Prepared for Nick Clifford
with Claude (Anthropic)

August 2026 · fourth revision

Abstract

`dotnet-rs` contains 581 unsafe blocks whose soundness rests on 620 prose `// SAFETY:` comments. This study evaluates a proposal to replace trust in that prose with machine-checked proof: a dependently-typed, Curry–Howard-style DSL in which each comment’s argument becomes a checkable term. Drawing on a survey of the repository’s unsafe corpus (which factors into approximately nine invariant families and 40–60 sites of genuine difficulty), the Rust verification literature (RustBelt, RefinedRust, VeriFast, Verus, Tree Borrows, RFC 3842 safety tags), and the state of managed-runtime formalization (no mechanized ECMA-335 model exists), we find the proposal feasible for a single developer with LLM-agent support, under one architectural correction: the DSL should be untrusted surface syntax elaborated into Lean 4 rather than a language with its own kernel. The recommended design is an embedded, tactic-oriented proof-script language in the lineage of `dotnetdll`’s `asm!` macro — block-shaped scripts wrapping each unsafe site, whose premises are zero-sized ghost tokens checked by `rustc` for existence, `cfg`-liveness, and dataflow — elaborating deterministically to statements over an executable Lean model of the ECMA-335 core, differentially tested against the project’s fixture oracle, paired with a small Miri-falsifiable axiomatization of Rust’s memory model. The verifier decomposes into independently versioned packages whose only contact with the VM is two build dependencies and two content-hashed manifest artifacts, so ordinary builds never invoke Lean. Two soundness defects introduced into the codebase shortly before this study — both instances of prose citing a witness that no code establishes — supply the empirical premise; under the token design one of them is not expressible. We give a staged roadmap (hygiene, embedded token layer, VES model, protocol theorem, site campaign) together with a decomposition into supervised agent-refactor work packages sized by checklist complexity and verification-signal quality; every stage terminates in an independently usable artifact, and the mechanized ECMA-335 core is publishable regardless of the proof campaign it enables.

Contents

1	Overview and principal findings	4
2	The system under study	5
2.1	Purpose and philosophy	5
2.2	The unsafe corpus, quantitatively	6
2.3	Two recent soundness defects	7

3	A taxonomy of the proof obligations	7
3.1	The families	7
3.2	Two irreducibly untrusted premise classes	9
3.3	Limits of Rust-level typing	10
4	The formal content of a <code>// SAFETY: comment</code>	10
4.1	The proposition behind an <code>unsafe block</code>	10
4.2	The role of the Curry–Howard correspondence	12
4.3	The specification side: an ECMA-335 abstract machine	12
5	Related work	13
5.1	Foundational semantics	13
5.2	Deductive verifiers for Rust	13
5.3	Structured safety comments: an emerging convention	15
5.4	Formal models of managed runtimes	15
5.5	Proof-assistant backends	16
6	Design space	16
6.1	Option A: a bespoke dependently-typed checker	16
6.2	Option B: an embedded script DSL elaborated into Lean 4	17
6.3	Option C: adopt an existing verifier wholesale	18
6.4	The two-level model architecture	18
6.5	Compartmentalization: packages, boundaries, artifacts	19
7	VesPROOF: an embedded proof-script DSL	22
7.1	Design lineage: the <code>asm!</code> pattern from <code>dotnetd11</code>	22
7.2	Surface syntax	22
7.3	Premises as ghost tokens	23
7.4	Binding, hygiene, spans, and <code>cfg</code>	24
7.5	Deterministic elaboration to Lean 4	25
7.6	Diagnostics	26
7.7	Worked example 1: the cross-arena rebrand (F1+F6)	26
7.8	Worked example 2: tracing completeness (F5+F2)	27
7.9	Worked example 3: a proof that fails at compile time (F4)	27
7.10	Treatment of template comments	27
8	Risks and limitations	28
8.1	The trusted computing base, itemized	28
8.2	The model-fidelity gap	28
8.3	Refactor churn under an LLM-agent workflow	29
8.4	The stop-the-world protocol theorem	29
8.5	Trusted-premise creep	29
8.6	Provenance	30
8.7	Cost model	30
9	Roadmap	30

10 Decomposition into supervised agent refactors	32
10.1 Sizing measures	32
10.2 Ordered work packages	33
10.3 Pipeline shape	35
11 Recommendation	35

1 Overview and principal findings

Principal finding

The proposal is feasible for a single developer working with LLM agents, subject to one architectural correction. A machine-checked proof layer over `dotnet-rs`'s `// SAFETY: corpus` is tractable because the corpus factors into approximately nine invariant families, 40–60 proof sites of genuine difficulty, and a small number of protocol theorems — not 620 independent obligations. The correction concerns the checker: the dependently-typed language should be a domain-fitted *surface* — an embedded, tactic-oriented proof-script DSL in the lineage of `dotnetdll`'s `asm!` macro, whose premises are ghost tokens checked by `rustc` and whose scripts elaborate into Lean 4 — rather than a calculus with its own kernel and solver. The fallback position named in the original proposal (“develop a model environment in Lean or Rocq and compile our proof language to that”) is, on the evidence assembled here, the correct primary design.

Six findings carry the argument:

1. **The premise is confirmed empirically in this repository.** Two soundness defects entered the tree in the five commits preceding this study — a feature-gated no-op cited as an alignment witness, and a Sync implementation justified by a false claim — and both instantiate a single failure mode: *prose asserting a witness that no code establishes*. Under the ghost-token design the first is not expressible — citing a `cfg`-excluded guard is a compile error — and the proof layer addresses the class (section 2.3, section 7.9).
2. **Each component has substantial precedent; the composition does not.** Machine-checked proofs of real unsafe Rust exist (RefinedRust, VeriFast, Verus); machine-readable safety annotations are an emerging Rust convention (RFC 3842, `tag-std`); verified collectors and verified bytecode machines exist (Hawblitzel-Petrunk, JinjaThreads). What exists nowhere is a connection between such proofs and a managed-runtime specification model: *no mechanized formalization of ECMA-335 exists*. That gap accounts for roughly half the projected cost and all of the novelty; an executable Lean model of the VES core would be a publishable artifact independent of the proof campaign (section 5).
3. **The verification problem is two-level by nature.** The difficult `// SAFETY: comments` appeal simultaneously to Rust-side facts (validity, aliasing, provenance under Tree Borrows) and VES-side facts (stop-the-world protocol state, layout faithfulness, evaluation-stack typing, GC reachability). The architecture must therefore pair a Rust memory-model fragment with an ECMA-335 abstract machine and a representation relation between them — the structure of a compiler-correctness proof, applied in the reverse direction (section 4, section 6.4).
4. **Lean 4 is the appropriate primary backend; Rocq is the appropriate escalation path.** Lean is stronger on the model half (executable specifications, differential testing against the existing 486-fixture oracle, elaborator metaprogramming for the DSL, the largest automation and agent-tooling ecosystem); Rocq holds the foundational Rust-semantics results (Iris, RustBelt, Tree Borrows, RefinedRust). The recommended design states Rust-substrate obligations against a small, Miri-falsifiable axiomatization in Lean and retains RefinedRust-in-Rocq as an upgrade path for a small set of critical modules (section 6.2).

5. **Compartmentalization determines survivability.** The verifier decomposes into five small Rust crates and five Lean packages, each separately versioned, publishable, and abandonable (section 6.5); the only artifacts crossing the boundary to `dotnet-rs` are two small build dependencies and two content-hashed manifests. The VM’s build and refactor velocity therefore never couple to Lean, and a stalled proof campaign freezes sites at `trusted` rather than breaking anything.
6. **Staging dominates ambition.** The roadmap (section 9) is cut so that every phase delivers an artifact the VM uses in CI within that phase: hygiene fixes and a blocking UB gate; the embedded token and script layer with `rustc`-checked premises; the executable VES model; the STW protocol theorem (the principal technical risk, section 8.4); and finally the site campaign under a trusted-premise ratchet. Section 10 refines the phases into an ordered sequence of supervised agent-refactor work packages sized by checklist complexity and verification-signal quality rather than by calendar time.

2 The system under study

2.1 Purpose and philosophy

`dotnet-rs` (HEAD 97e8f658 at the time of this survey) is an experimental Rust implementation of the ECMA-335 Virtual Execution System [13]: a 27-crate workspace (~75 kLOC of Rust) that loads real .NET assemblies and interprets CIL, with a `gc-arena`-based stop-the-world garbage collector extended by a custom multi-arena STW coordinator, a two-pass structured-exception-handling state machine, generics, reflection, P/Invoke via `libffi`, and a feature-gated multithreading runtime. The project’s self-description is deliberate: “an active research/runtime project focused on *correctness, architecture, and validation workflows*” — not performance, not production readiness.

That focus manifests as an unusually strong verification discipline for a codebase of this kind, which bears directly on feasibility: the proposal extends existing practice rather than introducing one.

- Two workspace-wide `clippy` lints at deny — `undocumented_unsafe_blocks` and `multiple_unsafe_ops_per_block` — so every unsafe operation is individually annotated, by policy.
- A written unsafe-code charter in `CONTRIBUTING.md`: SAFETY comments “must explain the invariant that makes the operation valid; ... not merely restate the operation”; layout transmutes require co-located size assertions; `unsafe impl Send/Sync` must be unconditional and paired with `assert_impl_all!` guards; `unsafe impl Collect` must justify every untraced field.
- Mechanized side conditions where the type system reaches: 52 `static_assertions/const asserts`, pinned layout-size test matrices across the feature cross-product, a compile-time lock-order DAG (`define_lock_order_dag!` with negative `AcquireAfter` assertions), and capability tokens with private constructors (`GcLifetime<'gc>`, `GcReadyToken<'ctx>`, `HeldLockLevel<L>`).
- Documented *accepted-risk boundaries*: the two places where the safety argument cannot be expressed through `gc-arena`’s API are isolated, named, and described in the GC and memory-safety design document, one of them carrying the codebase’s only `// DANGER: comment`.

Equally relevant are the deliberate compromises, because several of them become *axioms* any proof system must either import or discharge: the GC is currently non-moving (many raw-pointer proofs silently assume it); multi-arena collection is STW-only; release builds keep `overflow-checks =`

Measure	Count
<code>unsafe { }</code> blocks (all sources)	581
excluding tests/fuzz harnesses	542
<code>unsafe fn</code> declarations/definitions	99
<code>unsafe impl</code>	29
of which <code>Collect<'gc></code> (GC tracing)	22
of which <code>Send/Sync</code>	7
<code>// SAFETY:</code> comments	620
unique first lines	354
single-line / multi-line	363 / 257
# Safety rustdoc contracts on <code>unsafe fn</code>	62

Table 1: The unsafe corpus at HEAD 97e8f658.

`true`; strict-provenance Miri was evaluated and abandoned as infeasible (dependencies perform into-pointer casts), even though `expose_provenance` is load-bearing in the managed-pointer serialization; and the instrumentation gates — Miri (tree-borrows mode), Valgrind, all four fuzz targets — are continue-on-error in CI, i.e. *informative, never blocking*.

2.2 The unsafe corpus, quantitatively

Table 1 summarizes the corpus. Unsafe concentrates where one would expect: `dotnet-vm` (130 blocks), `dotnet-value` (114), `dotnet-runtime-memory` (69), `dotnet-utils` (67), then SIMD, threading intrinsics, and `P/Invoke`. By kind, the corpus is dominated by raw pointer arithmetic (~190 sites), raw dereference (~90), atomics constructed over raw memory (36 `AtomicUN::from_ptr` sites), `slice::from_raw_parts` (20), block copies (21), and a small number of high-consequence specials: 6 `gc-arena` brand escapes (`Gc::from_ptr`, `unlock_unchecked`), ~15 STW-only lock bypasses, 3 non-test transmutes, and the `libffi` call path. There are no unions, and only 3 test-only environment mutations.

Two structural facts about the *comment* corpus shape the entire feasibility question:

1. **The corpus is bimodal.** 342 of 620 comments (55%) share their first line verbatim with another comment; one template sentence appears 67 times in one file (`raw_memory_ops_impl/mod.rs`), deferring to “the enclosing raw-memory operation’s documented preconditions.” This mass is bookkeeping — near-zero proof content, discharged today by the lint, not by reasoning. The genuine multi-invariant arguments number roughly 40–60 and cluster in eight files: `object/mod.rs`, `heap.rs`, `thread_safe_lock.rs`, `statics.rs`, `storage.rs`, `access.rs`, `write_barrier.rs`, and `pinvoke/call.rs`. *A proof campaign should target those sites and the shared contracts behind the templates — not 620 individual proofs.*
2. **The hard premises are global, not local.** The load-bearing facts are protocol facts: “STW has stopped every mutator,” “the GC does not move objects,” “only the executor thread touches this state,” “off was recorded from this slot when the pointer was created.” A per-block, first-order annotation language cannot even state them; they require invariants over a model of the runtime (section 4).

2.3 Two recent soundness defects

The most direct empirical support for the proposal comes from the project’s recent history. Two soundness defects entered the tree in the five commits preceding this study, and both have the same shape: *a SAFETY comment asserted a fact that no code established.*

(a) The vacuous alignment guard. `validate_alignment` (`dotnet-utils/src/lib.rs:58-72`) compiles to a no-op unless the non-default `memory-validation` feature is on. Six call sites cite it as the alignment witness for atomic access. Four are saved by defense-in-depth (the atomic layer re-checks internally), but `StackValue::load_atomic/store_atomic` (`stack_value.rs:743-802`) proceed to `AtomicU32::from_ptr` with *no* alignment check in every shipping configuration — UB on a misaligned pointer, currently latent only because the live instruction paths happen to route through a different, guarded entry point. The stated proof at those sites is vacuously “discharged” by a function that does nothing.

(b) The false Sync premise. Commit `13d1edd4` added `unsafe impl<T: Send> Sync` for the single-threaded compat locks with the justification “single-threaded builds cannot create a second thread” — false (the build spawns a timeout-waiter thread), and in the conditional-impl form the project’s own charter forbids. It was reverted two commits later, but its narrower successor (`unsafe impl Send/Sync for SharedGlobalState`, `state.rs:346--355`) still asserts a whole-program discipline (“only the executor accesses VM state; the waiter touches only `abort_requested`”) that nothing checks — and the co-located `assert_impl_all!` *confirms the assertion rather than validating it.*

Both defects instantiate the failure mode a machine-checked proof layer exists to detect: prose citing a witness that is absent, disabled, or false. Neither Miri nor the fixture suite detected them, nor could they have — the relevant gates are non-blocking and the defective paths lie outside their coverage. This is the study’s empirical premise: the project has exhausted what convention, lints, and runtime assertions can deliver.

3 A taxonomy of the proof obligations

Formalizing 620 comments individually would be neither tractable nor useful. The corpus factors: roughly nine invariant *families* cover ~95% of the sites, and each family is a single predicate (or small theory) stated once and instantiated per site. This section catalogs the families, gives a representative site for the hard ones, and records — for each — which facts are Rust-level, which are VES-level, and why Rust’s type system alone cannot close the gap. The family structure is the raw material for the DSL of section 7.

3.1 The families

F1 — Rooting / STW liveness (~70 sites)

“This allocation cannot be freed (or moved) during this access.” *Rust facts*: a `Gc<'gc>` handle or lock guard in scope; an `Arc` refcount. *VES facts*: the STW protocol has parked every mutator; `unregister_arena` waits for `active_leases == 0`; arena generation stamps match; the GC is non-moving. *Gap*: the temporal premise is a global protocol state, invisible to types; the non-moving assumption is an unstated global axiom.

F2 — Layout faithfulness (~60 sites)

“The layout descriptor says offset k holds type X , so reading these bytes as X is valid.” *Rust facts*: slice bounds; bytes written by the matching serializer. *VES facts*: `FieldLayoutManager/GcDesc` faithfully model ECMA-335 field layout (§II.10.7, §I.8.5) for the concrete generic instantiation; the reference-overlap prohibition holds. *Gap*: the premise is a correspondence between computed Rust data and the standard’s layout rules — exactly the representation relation $\mathcal{R}$ of section 6.4. This family is uniquely consequential: `LayoutManager::trace` derives GC reachability from the same descriptor, so a layout defect is simultaneously a type confusion and a GC reachability error.

F3 — Evaluation-stack slot typing (~20 sites)

“This slot was pushed as X ; this raw view of it is valid, and recorded interior pointers survive `Vec` reallocation.” *Rust facts*: enum discriminants, `Vec` growth semantics. *VES facts*: eval-stack typing (§I.12.3.2.1); `PointerOrigin::Stack(StackSlotIndex)` indexes the live stack; every growth is followed by `apply_reallocation_fixup()`. *Gap*: a history-indexed (ghost) fact — “`off` was recorded from that slot when the pointer was created” — plus a whole-structure re-establishment obligation after every reallocation. The most amenable family to dependent indexing (`Slot  $\tau$`), and the one closest to Gordon–Syme’s stack-typing judgments [17].

F4 — Atomic access width/alignment (~40 sites)

“The pointer is valid and aligned for the selected atomic width, and no concurrent non-atomic access races it.” *Rust facts*: `align_of`, guard exclusion. *VES facts*: §I.12.6.2 alignment, §I.12.6.6 atomicity guarantees, the `unaligned.prefix`, and `.NET`’s asymmetric policy (misaligned `Interlocked` $\Rightarrow$ `DataMisalignedException`; misaligned volatile $\Rightarrow$ lock-guarded fallback). *Gap*: three discharge mechanisms coexist (an always-on guard, a feature-gated no-op, an internal re-check) and the prose does not say which one it is citing — the direct cause of defect (a) in section 2.3.

F5 — Tracing completeness (22 `unsafe impl Collect`)

“This trace visits every contained GC reference.” *Rust facts*: match exhaustiveness, field enumeration. *VES facts*: the root set (`ExecutionEngine` $\rightarrow$ `CallStack` $\rightarrow$ `ThreadContext` $\rightarrow$ `StackFrame`) is complete; continuation state (`VmContinuation`) covers all cross-safe-point values; cross-arena references are recorded rather than traced. *Gap*: a *completeness* (universally quantified, structure-directed) property — the classic GC obligation of the verified-collector literature [19], here on the mutator side. Mechanically checkable by reflection over type structure;

a derive-style audit already gets most of it, but the manual impls are exactly the risky ones.

F6 — gc-arena brand and token discipline (~10 sites)

“This 'gc-branded value cannot escape its arena / this token witnesses a mutation context.” *Rust facts*: invariant lifetimes, private constructors, `for<'a>` closure confinement. *VES facts*: `CallStack` lives in the arena root; cross-arena references never carry a brand. *Gap*: largely closed by Rust — this family is the clearest demonstration of what lifetime branding achieves — except for one deliberate violation: cross-arena tracing mints `Gc::from_ptr` with the *wrong arena*’s brand (`heap.rs:436--445`), justified by STW liveness together with non-escape. This is the most instructive single target for the DSL (section 7.7).

F7 — Immutable-field publication (~8 sites)

“owner_id is written once at construction, so this lock-free read is race-free.” *Gap*: needs immutability *plus* a happens-before edge publishing the initializing write; the prose supplies the former and leaves the latter implicit. A standard release/acquire obligation over the STW handshake.

F8 — Lock order and safepoint discipline

Partially mechanized already — the `define_lock_order_dag!` macro with negative compile-time assertions; the unmechanized residue is the four `GcScopeGuard` conventions (never poll a safepoint or allocate while holding a heap borrow, ...). *Gap*: a rely-guarantee protocol; violations become false premises for F1/F5 rather than independent UB.

F9 — Leaked-Box metadata lifetime (~14 sites)

“The `MetadataArena` solely owns these leaked allocations and outlives every descriptor into them.” Conventional ownership reasoning; fully expressible in existing separation logics; low risk, low novelty.

3.2 Two irreducibly untrusted premise classes

Two families of premise cannot be proved, only isolated, and the DSL must mark them with a distinguished trust class rather than obscure the distinction:

- **The managed program as adversary.** For `cpblk`, `initblk`, `Unsafe.*` and friends, ECMA-335 itself says behavior is undefined if the (unverifiable) IL supplies invalid operands (e.g. §III’s `cpblk` contract). The VM’s SAFETY comments correctly cite the spec contract as premise (`instructions/memory.rs:44--51`). Formally these become *conditional* theorems: *if* the input IL is verifiable (in the §III.1.8 sense) or the host accepts the unverifiable-code trust boundary, *then* the implementation is safe. A mechanized CIL verifier for the executed fragment would discharge the condition for verifiable inputs — a natural but strictly-later extension.
- **The FFI boundary.** `P/Invoke` ABI agreement between a `libffi` CIF and an arbitrary native library is not provable in any model that stops at the process boundary. The pinning and liveness-across-

the-call obligations *around* the FFI call are provable (F1); the call itself is an axiom per imported function.

3.3 Limits of Rust-level typing

The corpus confirms the intuition behind the proposal. Rust-level typing already carries what it can: branded lifetimes confine arena handles (F6), private-constructor tokens witness capabilities, the lock DAG is compile-time, newtype indices name their domains. What remains is precisely what an ML-family type system without dependent types, quantified invariants, or ghost state cannot say:

- *Value-dependent facts*: “ $\text{offset} + \text{size} \leq \text{len}$,” “aligned to `align_of::<u32>()`” — refinement-type territory (the project’s own newtype policy explicitly declines to carry bounds, pushing every range obligation to the use site).
- *Temporal/protocol facts*: “all mutators are parked for the duration of this read” — a statement about *other threads*’ continuations, requiring ghost protocol state (Iris-style) or a global LTS refinement.
- *History facts*: “this offset was recorded from that slot at pointer-creation time” — ghost variables over execution history.
- *Correspondence facts*: “GcDesc agrees with §II.10.7 layout for this instantiated type” — a relation between runtime data and a specification-level function, i.e. a theorem about two models at once.
- *Completeness facts*: “every GC reference in this structure is visited” — universally quantified over a type’s recursive structure.

Encoding any one of these in stable Rust’s traits-and-lifetimes fragment is the “many layers of complex type logic” the proposal anticipates: achievable in isolated cases (tpestate, const generics, sealed tokens), unmaintainable as a system, and in any case incapable of expressing the ECMA-335 correspondence. The conclusion of this section is not that Rust’s guarantees are weak — the corpus shows the project extracting a great deal from them — but that the residual obligations are precisely of the kind for which proof assistants exist.

4 The formal content of a `// SAFETY: comment`

Before we can evaluate designs, we must pin down the mathematical content of the thing being proposed. This section fixes terminology used in the rest of the study: what proposition an `unsafe` block’s safety comment actually denotes, what a proof of it would range over, and where Curry–Howard enters.

4.1 The proposition behind an `unsafe` block

Rust’s soundness story is often summarized as: *safe code cannot cause undefined behavior*. The precise version, due to RustBelt [24], is a *semantic typing* statement. Each type τ is interpreted not as a syntactic classifier but as a predicate $\llbracket \tau \rrbracket$ on values — in RustBelt’s case, a predicate in the Iris separation logic [25] over a low-level operational semantics of MIR. A function is *semantically well-typed* when it maps arguments satisfying its parameter predicates to results satisfying its return predicate, without ever executing an undefined operation, *regardless of what well-typed code surrounds it*. The fundamental theorem says syntactically well-typed (safe) code is semantically well-typed; soundness of a program containing `unsafe` then reduces to a sequence of manual lemmas:

Definition 4.1 (Safety obligation of an unsafe block). Let B be an `unsafe` block occurring in a function f of module M , and let I_M be the module invariant — the (possibly ghost-state-indexed) predicate relating M ’s private data structures to the abstraction M exposes. The safety obligation of B is the Hoare-style statement

$$\{ P_B * I_M \} B \{ Q_B * I_M \}$$

where P_B is what the surrounding safe code is known to have established at entry to B (from types, flow, and prior checks), and Q_B is what the code after B relies on. “Proving the SAFETY comment” means exhibiting a derivation of this triple against a formal operational model of the language, in which every primitive step of B is shown to satisfy its side conditions (validity, alignment, aliasing, initialization, ...).

Three consequences of this framing structure the rest of the study.

(1) The obligation is relative to two semantics. The side conditions in Definition 4.1 come from Rust’s dynamic semantics — pointer validity and provenance as fixed by the operational aliasing model (Stacked Borrows [23], now superseded in practice by Tree Borrows [42]) — but the *preconditions* P_B in `dotnet-rs` are frequently facts about the *virtual machine being implemented*, not about Rust: “all mutator threads are stopped,” “this object’s arena is registered and its generation matches,” “the layout descriptor says offset k holds an object reference.” Those facts are only expressible against a model of the VES itself — an abstract machine for a fragment of ECMA-335 [13]. A verifier for `dotnet-rs` safety comments is therefore necessarily a *two-level* system:

$$\underbrace{\text{ECMA-335 abstract machine model}}_{\text{gives meaning to } P_B, Q_B, I_M} \implies \underbrace{\text{Rust/MIR memory model}}_{\text{gives meaning to the steps of } B}$$

with a simulation relation connecting VM-level states to the Rust data structures that represent them. This is the same architecture as a compiler correctness proof (CompCert [34], CakeML [29]), run in reverse: instead of proving a compiler preserves source semantics, we prove a hand-written interpreter’s internal transitions refine the specification’s transitions, and that its `unsafe` shortcuts are justified by invariants the specification-level model maintains.

(2) Module invariants, not block-local facts, carry the weight. Almost none of the interesting `// SAFETY`: comments in `dotnet-rs` are self-contained. A comment like “STW has stopped every mutator, so the owning arena cannot be freed during this read” (section 3) appeals to a *protocol*: a state machine over coordinator phases, arena registration, lease counts, and thread states, whose transitions are scattered across several crates. In separation-logic terms this is ghost state governed by an invariant I_{STW} ; individual blocks open the invariant, use a token certifying the current phase, and close it. Any DSL that can only state block-local, first-order facts (“ p is aligned,” “ $n \leq \text{len}$ ”) will be able to discharge the easy third of the corpus and will hit a wall exactly where the risk is concentrated. This is why the study devotes section 6 to the question of *where the invariant layer lives*, and it is the single strongest argument for reusing an existing Iris-class logic rather than inventing a bespoke solver.

(3) The proof is about a model of the code, not the code. Unless the checker consumes the actual MIR of the crate (as RefinedRust [16] and Aeneas [20] do), there is a formalization gap: someone (or some tool) must translate each `unsafe` block and the definitions it touches into the proof language, and the fidelity of that translation is part of the trusted computing base. Section 8 quantifies this: for a codebase undergoing LLM-assisted refactors at `dotnet-rs`’s pace, an untethered model drifts from the code within weeks. Mechanized tethering — extraction from MIR, or at minimum syntactic anchoring

with CI-enforced drift checks in the style of the project’s existing `check_doc_drift.sh` — is not optional polish; it is what distinguishes verification from documentation.

4.2 The role of the Curry–Howard correspondence

The Curry–Howard correspondence identifies propositions with types and proofs with programs: a term $t : \Phi$ is a checkable certificate of Φ . For this project the correspondence matters in a specific, practical way: it determines the *artifact format*. A `// SAFETY:-comment` DSL under Curry–Howard is a language of *proof terms* whose types are safety propositions; the checker is a type checker; and soundness of the whole edifice reduces to (a) soundness of the ambient type theory and (b) adequacy of the axioms/model against which the propositions are stated. Concretely, three design families instantiate this:

Deep proof terms, custom checker. The DSL is itself a small dependently-typed calculus (Π, Σ , indexed inductive families for the invariant taxonomy of section 3), with a hand-written kernel. Curry–Howard is used directly. The kernel joins the TCB; decidability and totality of type checking must be engineered (bidirectional checking, no unrestricted conversion); and every automation convenience (arithmetic, congruence) must be built, or the proofs become impractically manual. Bespoke kernels developed alongside large host projects have a poor historical record; section 6.1 argues this option is dominated.

Shallow embedding into an existing proof assistant. Propositions and proofs are ordinary Lean 4 [37] or Rocq [41] terms; the annotation beside the `unsafe` site — whether an attribute, a comment, or an embedded proof script — determines the statement and sketches the proof, while the checked artifact lives in the host. Curry–Howard is inherited from the host’s kernel, which is small, battle-tested, and independently checkable. The VM model, the Rust memory-model fragment, and the simulation relation are libraries in the host. This is the architecture of RustBelt (Iris/Rocq), RefinedRust (Iris/Rocq, with a MIR front end), and Aeneas (extraction to Lean).

SMT-mediated verification conditions. Verus [30] and Creusot [12] compile annotations to SMT queries. This buys enormous automation for first-order obligations (bounds, alignment, arithmetic) at the cost of the proof-term ideal: an SMT “unsat” is not a checkable certificate in the Curry–Howard sense, and quantifier-heavy protocol invariants strain the paradigm. The pragmatic middle ground — proof terms for protocol lemmas, SMT for leaf arithmetic — is exactly the division of labor RefinedRust and Verus each approach from opposite ends.

4.3 The specification side: an ECMA-335 abstract machine

The second layer — the one that distinguishes this proposal from general unsafe-Rust verification — is a formal model of the VES fragment that `dotnet-rs` implements. The standard itself is prose, but it is unusually formalization-friendly prose: it specifies an explicit evaluation-stack machine, a typed memory store with a precise alignment and atomicity contract (§I.12.6.2, §I.12.6.6), an acquire/release volatile semantics (§I.12.6.7), a pointer taxonomy separating managed from unmanaged pointers with GC-reporting obligations (§II.14.4, §III.1.1.5), layout rules including the prohibition on object references overlapping other fields (§II.10.7), and finalization lifecycle rules (§I.8.9.6.7). Prior mechanizations of comparable scope exist and calibrate the effort: Fruja’s ASM-based CLR model covering the type system and bytecode verifier [15], Gordon and Syme’s type soundness for a CIL fragment [17], Kennedy and Syme’s formalization of CLR generics [27], and — closest in spirit to what `dotnet-rs` needs — the

Jinja/JinjaThreads line in Isabelle/HOL, which grew from a sequential Java-like machine to a verified multithreaded JVM with a verified compiler over roughly a decade [28, 35].

The model does *not* need to cover the whole standard to be useful. The invariant families of section 3 touch a compact core: object and array layout, the managed-pointer discipline, the memory/atomicity model, GC reachability and finalization, and the STW protocol (which is an implementation construct *justified* by the standard’s GC latitude, not mandated by it). A model of that core, indexed by the layout function that `dotnet-rs`’s `LayoutManager` actually computes, is the central artifact; everything else in the roadmap of section 9 is staged against it.

5 Related work

No existing system realizes this proposal end-to-end, but every component has been built somewhere, and the most reliable cost evidence for each comes from its builders. This section surveys four bodies of work: foundational semantics for `unsafe Rust`, deductive verifiers for Rust, the “specification language in comments” tradition, and formal models of managed runtimes. Table 2 summarizes the Rust-side tools against the axes that matter here.

5.1 Foundational semantics

RustBelt [24] established the template this whole proposal rests on: interpret each type as a separation-logic predicate in Iris [25], prove safe code well-typed once and for all, and admit each `unsafe` library by *manually proving its implementation inhabits the semantic interpretation of its API*. That is the formal content of a `// SAFETY: comment`. The qualification is equally instructive: RustBelt’s proofs concern λ_{Rust} , a core calculus, not surface Rust, and each verified library (`Cell`, `Mutex`, `Arc`, ...) cost an expert person-months of work in Rocq.

Two further facts about the foundations constrain any design:

- **The aliasing model is part of the obligation.** A proof that an `unsafe` block is memory-safe but violates the compiler’s aliasing discipline proves the wrong theorem: `rustc` optimizes under Stacked Borrows [23] and, increasingly, Tree Borrows [42] (PLDI 2025 Distinguished Paper, now formalized in Rocq and implemented in Miri, and already the mode `dotnet-rs`’s CI runs). Any DSL that cannot state `retag`/permission obligations silently under-approximates UB.
- **Rust has no complete normative formal semantics in 2026.** The Ferrocene specification was adopted as the seed of the official Rust specification in March 2025 [39], but it is prose; `a-mir-formality` [36] covers static semantics only; MiniRust [26] is the leading candidate for the operational story and is explicitly the UB list a proof should target. Every “real theorem about the Rust substrate” is therefore stated against a best-available approximation, and the study’s soundness claims must say so.

5.2 Deductive verifiers for Rust

Three entries warrant elaboration because they delimit the design space.

RefinedRust is the proposal, minus the ECMA-335 layer. RefinedRust [16] attaches refinement-type annotations to real Rust source, translates the actual MIR into a Rocq model (Radium), and discharges obligations with untrusted-but-foundationally-checked automation inside Iris; it verified a `Vec`

Tool	Handles unsafe?	Foundation	Automation	Annotation site
RustBelt [24]	yes (λ_{Rust})	Iris/Rocq, foundational	none	external Rocq
RefinedRust [16]	yes (real MIR)	Iris/Rocq, foundational	high (Lithium)	<code>#[rr::...]</code> attributes
VeriFast [38]	yes	SL, tool-trusted [†]	symbolic exec.	<code>/*@...@*/</code> comments
Verus [30]	re-encoded [‡]	SMT; Iris justification [18]	very high (Z3)	<code>verus! {}</code> macro
Creusot [12]	emerging	Why3/Coma	high (portfolio)	attributes
Gillian-Rust [4]	yes (hybrid)	SL, prototype	high	attributes
Aeneas [20]	no	Lean 4 (pure translation)	Lean tactics	external Lean
Prusti	no	Viper	high	attributes
Flux [33]	no	liquid types/SMT	very high	attributes
Kani [3]	bounded only	CBMC	total	harnesses

Table 2: Rust verification tools relevant to the proposal. [†]Foundational certification of VeriFast results is in progress via hinted mirroring [21]. [‡]Verus does not verify preexisting unsafe blocks; code is rewritten against trusted linear-permission APIs (`PointsTo`, `PPtr`, `PCell`).

variant including intricate raw-pointer code. It demonstrates that annotation-driven, machine-checked, foundational proofs about real unsafe Rust exist today. It also supplies the clearest cost evidence: it was built by a team of Iris experts, language coverage remains partial, and difficult cases still require manual Iris lemmas. Wholesale adoption by a single developer would itself be a research project; adopting its *architecture* — annotations, a mechanized model, and foundationally checked automation — at narrower scope is the realistic variant.

VeriFast is the precedent for proofs that live in comments. VeriFast for Rust [38, 22] puts contracts, predicate definitions, and proof steps (lemma calls, `open/close`) in `/*@ ... @*/` comments beside the code, verifies modules containing unsafe blocks by symbolic execution, derives contracts for safe functions from RustBelt’s type interpretation, and — critically for this proposal — recent work adds Tree Borrows compliance obligations. It is one of the tools in the AWS `verify-rust-std` campaign [2] and verified properties of `std`’s `LinkedList`. Its trust story (an unverified checker core, with foundational certification arriving via hinted mirroring [21]) is exactly the trust position a custom `// SAFETY: checker would occupy` — a point section 6 leans on.

Verus marks the automation ceiling. Verus [30, 31] demonstrates what SMT combined with linear ghost permissions can achieve: systems code with pointer manipulation verified at industrial scale (Amazon, Microsoft), with proofs written in Rust syntax — and, since VerusBelt [18], an Iris/Rocq semantic soundness proof backing its permission axioms. Its limitation for `dotnet-rs` is structural: Verus is designed to own the code under verification. One does not annotate an existing `gc-arena`-based heap; one rewrites it against `PPtr/PCell` tokens. Verus is also the ecosystem with the most evidence for LLM-generated proofs (AutoVerus, SAFE), which is relevant to the intended workflow.

Kani [3] and Miri sit below all of these: bounded model checking and dynamic interpretation respectively. They prove no theorems, but they falsify candidate invariants cheaply, and the `verify-rust-std` campaign [2] (16,748 auto-generated harnesses; contract attributes now upstreamed into nightly `std` as an experiment) is the best available evidence on what annotating unsafe Rust at scale costs in practice.

5.3 Structured safety comments: an emerging convention

The Rust community is independently converging on machine-readable `// SAFETY: comments`. RFC 3842 “Safety Tags” [46] (opened July 2025, still open) proposes `#[safety::requires(tag, ...)]` on unsafe functions and `#[safety::checked(tag, ...)]` at call sites; the `tag-std` project [1] developed the underlying taxonomy and a linter that checks *structural consistency* — every obligation named by the callee is acknowledged by the caller — and applied it to `libstd`, covering 96.1% of public unsafe APIs. The authors note explicitly that tags “could support formal verification, e.g., by being extended into contracts.”

These systems check bookkeeping, not truth: a tag asserts that someone claims alignment holds, not that it does. That is precisely the gap a proof layer fills, and it fixes the surface syntax question: the DSL should be an *extension of* the emerging tag convention (tag = theorem name; proof term or proof-script reference attached per tag), not a fork of it. The older traditions — ACSL/Frama-C [5] on C and JML/OpenJML [10] on Java — are the existence proof for comment-carried, machine-checked specifications at industrial scale, and they teach two lessons: the WP-style split (SMT for the bulk, proof assistant for the residue) is what makes the burden bearable, and comment-hosted languages drift from the host language unless the compiler sees them — which favors attribute/macro embedding for anything load-bearing.

5.4 Formal models of managed runtimes

The specification-side picture is starkly asymmetric to the Rust side: **no mechanized formalization of ECMA-335 exists**. What exists is pre-mechanization-era and pen-and-paper:

- **Fruja’s ASM model** [15, 8] — the most complete formal CLR model ever built: modular Abstract State Machines covering the CIL instruction set, the exception mechanism (including filter/finally/fault interactions `dotnet-rs` implements as a state machine), and the bytecode verifier, with type-safety proofs. Dormant since ~2007, never mechanized. As a *source text* for a mechanization it is invaluable: the semantic edge cases are already worked out.
- **Gordon and Syme** [17] — type soundness for “Baby IL,” the theory behind the real verifier, defining the evaluation-stack typing and managed-pointer discipline that several of `dotnet-rs`’s invariant families instantiate.
- **Yu, Kennedy, and Syme** [44, 27] — soundness for the CLR’s generics implementation strategy (shared instantiations, runtime type representations); the invariant catalog for any theorem about `dotnet-rs`’s generic-instantiation caches.
- **Benton** [6] — a compositional program logic for a CIL-like stack machine; the early precedent for Hoare-style reasoning at the evaluation-stack level.

The JVM side shows what mechanization costs and what it buys. Jinja [28] and JinjaThreads [35] grew, over roughly fifteen years of Isabelle/HOL work, into a verified multithreaded Java-like machine with a verified compiler and executable semantics — proof that a full managed-bytecode machine is mechanizable, and a warning that it is among the largest artifacts in its proof assistant’s library. JSCert [7] contributes the methodology this study adopts for the ECMA-335 model: clause-by-clause correspondence between the prose standard and the mechanized definitions, plus an *executable* reference interpreter differentially tested against real test suites — a philosophy `dotnet-rs` already practices with its fixture harness against the real .NET runtime.

For the GC specifically, the verified-collector literature supplies the invariant vocabulary. Hawblitzel and Petrank [19] verified practical copying and mark-sweep collectors with Boogie/Z3 — two people, SMT-automated, performance-competitive — and their mutator-collector interface contracts (reachability abstraction, forwarding invariants) transfer directly to the obligations `dotnet-rs` discharges by hand around `gc-arena`. CakeML’s verified generational collector [40] and the rely-guarantee verification of a concurrent collector [45] mark the high end; Verve [43] shows a whole runtime under a typed managed language verified with SMT-heavy tooling. The relevant reading for `dotnet-rs` is mutator-side: the project does not implement a collector, it *uses* one, so its obligations are correct usage of `gc-arena`’s rooting/tracing contract — which is precisely the shape of the Hawblitzel-Petrank interface contracts.

5.5 Proof-assistant backends

The two candidate backends the author named are both viable, with sharply different prior art:

Rocq. Renamed from Coq at version 9.0 (March 2025) [41]. Every foundational unsafe-Rust result lives here: Iris [25], RustBelt, the Tree Borrows mechanization, RefinedRust, VerusBelt. If theorems must genuinely bottom out in the Rust aliasing model, Rocq is the only backend where that model already exists mechanized. Translation tools exist (`rocq-of-rust` [14] via THIR; `hax` [11] for a panic-free safe subset).

Lean 4. The stronger *engineering* ecosystem circa 2026 [37]: best-in-class metaprogramming for building a DSL elaborator, rapidly improving automation (`omega`, the `grind` tactic since v4.22, `lean-smt`), Mathlib as background library, and by far the largest LLM-proof corpus — directly relevant to this project’s agent-driven workflow. Aeneas [20] targets Lean as its primary backend (safe code only). An active Iris port exists (`iris-lean` [32]: MoSeL, UPred, invariants, later credits), but it is a port of the *logic* — there is no mechanized Rust semantics in Lean today.

The resulting design fork, developed in section 6: Rocq for the Rust-substrate half, Lean for the ECMA-335-model half, or one backend with the other half stated against a trusted axiomatization.

6 Design space

The proposal has four separable decisions: (1) where the proofs are *checked* (custom kernel, proof assistant, SMT, or a tiered mix); (2) where the proofs *live* (comments, attributes, an embedded script language, or an external proofs workspace keyed to the code); (3) what the propositions are *about* (a Rust memory model, an ECMA-335 abstract machine, and the simulation between them); and (4) how the pieces are *compartmentalized* into independently buildable, versionable, and abandonable packages. The decisions interact, but they fail independently, so we treat them in turn.

6.1 Option A: a bespoke dependently-typed checker

The reading of the proposal closest to its letter: design a small dependently-typed calculus — Π/Σ types, indexed inductive families for the invariant taxonomy of section 3, definitional equality strong enough for layout arithmetic — and implement its kernel as a new workspace crate, with proof terms carried beside the code and checked in CI.

The attraction is real: the calculus can be *domain-fitted*. Judgments like $\Gamma \vdash \pi : \text{rooted}(o)$ or $\Gamma \vdash \text{stw} \Rightarrow \text{live}(a)$ become primitive forms; proof terms stay short because the type theory knows

about arenas, layouts, and phases. And a kernel for a bidirectional dependently-typed checker is a known quantity — a few thousand lines for the core.

The case against is decisive at this project’s scale, and it is worth being explicit because it answers the question as asked:

1. **The kernel joins the TCB with no independent audit.** A bespoke checker occupies the same trust position VeriFast occupied for fifteen years [21] — “checked by a tool whose soundness argument is on paper” — except VeriFast has had thousands of users shaking it. Lean and Rocq kernels are small, ancient, and independently re-implemented; nothing built here in a year approaches that.
2. **Automation is the actual cost center, not the kernel.** The corpus is dominated by obligations with heavy arithmetic and layout content (offset/alignment computations, bounds, descriptor interning). In Lean these fall to `omega/grind/lean-smt`; in Rocq to `lia` and Iris’s proof mode; in a bespoke system every one of these must be rebuilt or the proofs become unwritable. The history of ACSL [5] says the SMT-for-the-bulk / assistant-for-the-residue split is what makes annotation burden bearable; a bespoke kernel starts with neither half.
3. **The propositions still need a model.** Even a perfect kernel proves theorems only relative to axioms about Rust and the VES. Those axiomatizations are the hard, novel work (section 6.4); they are needed under every option, and under Option A they get the least scrutiny.
4. **It forfeits the ecosystem the workflow depends on.** The author’s stated workflow is LLM-agent-driven. Lean 4 has the largest machine-proof training corpus and tooling surface of any assistant; a private calculus has zero. The single biggest practical lever — agents drafting and repairing proofs under a checker’s supervision — works dramatically better against a mainstream target.

Assessment of Option A

Building the calculus is feasible; trusting it is not justifiable, and its automation alone would exceed the budget of a reasonable first campaign. A domain-fitted *surface language* is nonetheless the right idea — section 7 designs one, including a tactic layer — but as untrusted syntax elaborated into an existing kernel’s terms, never as a kernel. The tactic point deserves emphasis, because it is where the bespoke-kernel temptation recurs: domain tactics (“open the STW invariant,” “frame this token”) are implemented as Lean metaprograms whose *output* the kernel checks, so the surface can grow arbitrarily specialized while the TCB does not grow at all.

6.2 Option B: an embedded script DSL elaborated into Lean 4

Here the DSL is a block-shaped, tactic-oriented proof script embedded in the Rust source as a macro that *wraps* each `unsafe` site — statement-shaped lines in the lineage of `dotnetdll`’s `asm!` macro [9] (section 7.1) — naming a goal (declared or inferred), acquiring premise witnesses as zero-sized Rust *ghost tokens* the compiler itself checks, and sketching the discharge with vocabulary tactics. The scripts elaborate deterministically into ordinary Lean 4 statements in a proofs workspace, where the proofs are ordinary Lean terms and the checker is Lean’s kernel. A small external tool bridges the two worlds by comparing content-hashed manifests — never by running Lean inside the VM’s build (section 6.5).

This is the architecture this study recommends, and the backend choice follows from section 5.5:

- **Lean 4 as the home** for the ECMA-335 abstract machine (executable, JSCert-style, differentially tested against the existing fixture oracle), for the invariant-family library and its tactic layer, and for the simulation-relation theorems. Lean’s metaprogramming carries both ends of the DSL: the vocabulary is compiled from one declarative source into Rust token types and Lean declarations, and the surface tactics are Lean tactic macros.
- **Rust-substrate obligations stated against a trusted axiomatization** of the MiniRust/Tree-Borrows UB rules — a Lean library of axioms (or definitions validated by testing) rather than a from-scratch re-mechanization of what already exists in Rocq. The alternative — doing the whole project in Rocq to sit on Iris/RustBelt/Tree Borrows — buys foundational depth on the Rust side at the cost of the model half (no executable-spec tradition as strong, weaker agent tooling, and the VES model is the bigger artifact). Section 8 treats the residual gap honestly: theorems about the Rust side are then conditional on the axiomatization’s fidelity.
- **An escape hatch upward:** for a handful of critical modules (the gc-arena boundary, the STW protocol), RefinedRust in Rocq remains available as a second campaign; nothing in the Lean-first design blocks it, because the script layer is checker-agnostic by construction (the goal names the obligation; which backend discharges it is a per-obligation choice recorded in the proof manifest).

6.3 Option C: adopt an existing verifier wholesale

For completeness: pointing RefinedRust, VeriFast, or Verus at dotnet-vm as-is.

- *RefinedRust* is the right theorem but the wrong scale today: partial language coverage meets a 26-crate workspace built on gc-arena’s branded lifetimes, GATs, and heavy trait towers; and the ECMA-335 layer would still have to be built inside its Rocq setting.
- *VeriFast* comes closest in form factor (proofs in comments, modules with `unsafe`, Tree Borrows obligations) and is the most credible off-the-shelf pilot: worth a two-week spike on dotnet-utils’ atomic-access module. Its separation logic knows nothing of the VES, so spec-level invariants become user-defined predicates — workable, unassisted.
- *Verus* would mean rewriting the heap and stack cores against its permission tokens — a different project (arguably a different, interesting project: a Verus-native VES), not a verification layer on this one. Its ghost-permission idiom is, however, the direct inspiration for the ghost-token layer of section 7.3, reused here at the annotation layer rather than as a rewrite.

Option C fails as the whole answer for the same reason it succeeds as a component: these tools verify Rust against Rust’s model. The author’s actual ambition — “take our ECMA-335 target, combine it with our Rust substrate, and show compliance” — requires the two-level structure that none of them ships.

6.4 The two-level model architecture

Whatever discharges the proofs, the propositions need the following mechanized stack; this is the load-bearing engineering of the whole proposal.

1. $\mathcal{M}_{\text{VES}}$ — **an executable ECMA-335 abstract machine in Lean 4**, covering the invariant-relevant core: the typed memory store with alignment/atomicity rules (§I.12.6.2, §I.12.6.6–§I.12.6.8), object/array layout including the reference-overlap prohibition (§II.10.7, §II.22.8), the

managed/unmanaged pointer taxonomy and GC-reporting rules (§II.14.4, §III.1.1.5), evaluation-stack typing for the executed CIL fragment (after Gordon–Syme [17]), reachability/finalization (§I.8.9.6.7), and — as an implementation-level module, since the standard only licenses it — the STW/arena protocol as a labelled transition system. Executability is non-negotiable: it is what lets the existing differential fixture harness test the *model* against real .NET, JSCert-style [7], before any proof depends on it.

2. $\mathcal{M}_{\text{Rust}}$ — **a shallow Lean fragment of the Rust value/memory model** sufficient to state UB side conditions: typed byte ranges, provenance-carrying pointers, validity/initialization predicates, data-race conditions, and the Tree Borrows obligations for the retag patterns the codebase actually uses (a deliberately small, test-validated axiomatization per section 6.2).
3. $\mathcal{R}$ — **the representation relation** connecting the two: layout descriptors computed by Layout-Manager correspond to $\mathcal{M}_{\text{VES}}$ layouts; `ObjectRef/ObjectPtr/ManagedPtr` values represent spec-level references; arena/coordinator states refine the protocol LTS. Each invariant family of section 3 becomes a predicate stated once over $\mathcal{R}$ and instantiated by many `// SAFETY`: sites.

The payoff structure matters: $\mathcal{M}_{\text{VES}}$ is useful *before any unsafe block is verified* — as a differential oracle and as the first mechanized ECMA-335 model, a publishable artifact in its own right. The proposal should be staged so that this asset lands first (section 9).

6.5 Compartmentalization: packages, boundaries, artifacts

The proposal’s suggestion to spin the verifier into its own workspace is realized as two sibling repositories: `ves-proof`, whose Rust packages live under `crates/`, and `ves-proof-lean`, whose independent Lake packages live under `packages/`. This concrete split generalizes to the design rule on which the project’s viability rests: *every piece is a separately versioned package; the only things crossing package boundaries are code dependencies pointing in one direction or serialized artifacts with versioned schemas; and dotnet-rs’s ordinary build never invokes Lean*. Table 3 lays out the decomposition; fig. 1 the dependency direction.

The two artifact schemas. Boundaries are serialization contracts, versioned like any other API:

- *Obligation manifest* (`obligations.v1.json`; producer `ves-check extract`, consumers `VesCore`’s stub generator and the structural rules): `per (site, cfg-set)` — goal term, premise list with trust classes, tactic sketch, source span, statement hash, subject hash, vocabulary and grammar versions. Canonical JSON (sorted keys, normalized paths, no timestamps) so the same tree always serializes byte-identically.
- *Proof manifest* (`proofs.v1.json`; producer `DotnetRsProofs`’ Lake export, consumer `ves-check` in `dotnet-rs CI`): `obligation hash` → theorem name, sorry-free flag, transitive axiom set, Lean toolchain, vocabulary version. Published as a tagged release artifact; `dotnet-rs` pins a version exactly as it pins a crate.

Compatibility rules. The vocabulary version is the compatibility anchor: statement hashes embed it, both manifests carry it, and `ves-check` refuses to compare manifests across a vocabulary major version. Additive vocabulary changes (new goal constructors, new tactics) are minor and never invalidate existing hashes; changing a constructor’s elaboration is major by definition because it moves hashes.

Package	Kind	CI owner	Role and independence
ves-syntax	Rust lib	ves-proof	script grammar, AST, canonical serializer, hashing; no proc-macro, no Lean; the <i>single</i> parser linked by both macro and checker
ves-vocabulary	data + codegen	ves-proof	declarative goal/premise/tactic source; generates ves-tokens types <i>and</i> VesCore declarations; semver'd; the compatibility anchor
ves-tokens	Rust lib (tiny)	ves-proof	ZST ghost tokens, cfg-mirrored constructors, TrustGrant; no_std, zero-cost; the only runtime dependency dotnet-rs gains
ves-macros	proc-macro	ves-proof	proof!, proof_impl!, contract!; parsing via ves-syntax; expansion = tokens + verbatim subject
ves-check	CLI bin	ves-proof	extraction, structural rules, manifest diff, ratchet budgets; needs no Lean toolchain
VesCore	Lean (Lake)	ves-proof-lean	vocabulary mirror, goal elaborator, statement hashing, ves_sketch tactic frame
VesModel	Lean (Lake)	ves-proof-lean	$\mathcal{M}_{\text{VES}}$, executable; differential test harness against .NET fixtures
RustAssumptions	Lean (Lake)	ves-proof-lean	$\mathcal{M}_{\text{Rust}}$ axioms + Miri-falsifier index
VesProtocol	Lean (Lake)	ves-proof-lean	STW/arena LTS, protocol invariants and theorems
DotnetRsProofs	Lean (Lake)	ves-proof-lean	$\mathcal{R}$ + site lemmas; exports the proof manifest
dotnet-rs integration	workspace changes	dotnet-rs	scripts at unsafe sites; dev-deps on ves-macros/ves-tokens; xtask ves-check CI job; TRUSTED.toml

Table 3: The package decomposition. The five Rust crates exist under ves-proof/crates/; the five Lean packages exist under ves-proof-lean/packages/. Each is publishable and testable in isolation.

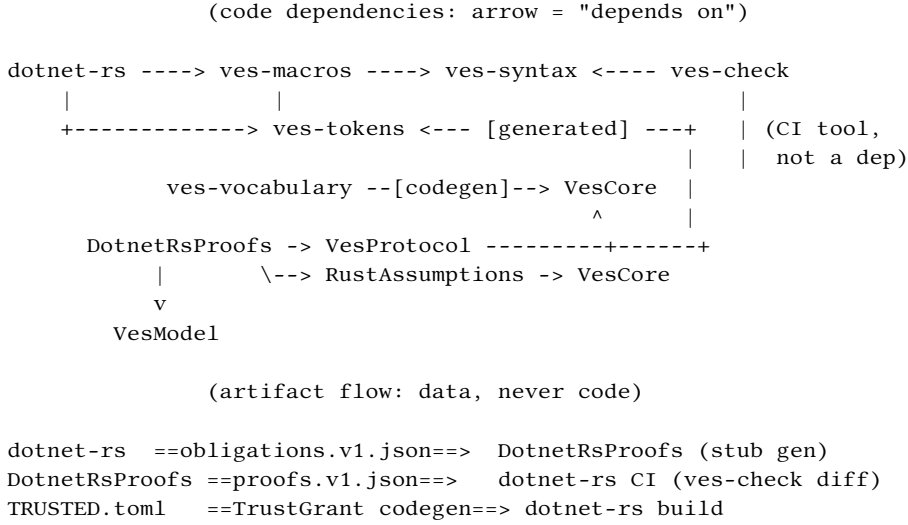


Figure 1: Dependency direction and artifact flow. No Lean package depends on `dotnet-rs` source; `dotnet-rs` depends on no Lean package; the two meet only in serialized manifests.

Proof manifests may lag the code: an obligation with no matching proof entry is *unproved* (ratcheted, warning-level by default; blocking for sites the policy designates critical), while structural violations — unwrapped unsafe, unregistered trust grants, budget breaches — always block. This split is what lets `dotnet-rs` refactor at full speed against a week-old proof manifest without ever weakening the structural floor.

CI ownership. `dotnet-rs`’s CI gains one fast job: `build` (which already type-checks every ghost token in every matrix leg) plus `ves-check` against the pinned proof manifest — seconds, no Lean. The `ves-proof` repository owns Rust tooling and its tests; `ves-proof-lean` owns Lake builds, the `VesModel` differential suite against real .NET, and the Miri falsifier index for `RustAssumptions`. A scheduled verifier job may coordinate both repositories, pull `dotnet-rs` HEAD, re-extract obligations, and open a drift report when hashes have moved — the cross-repository analogue of `check_doc_drift.sh`.

Independence and abandonability. Each layer is useful if everything above it stops: vocabulary + macros + tokens alone give rustc-checked premise discipline with zero Lean (Phase 1); `ves-check` with an empty proof manifest is a tag-std-class [1] structural layer; `VesModel` is a publishable mechanized ECMA-335 core with no dependence on the DSL at all; `VesProtocol` and `RustAssumptions` are independently meaningful theories; only `DotnetRsProofs` is coupled to the VM, and abandoning it merely freezes every site at `trusted` — strictly better than today’s prose, at no ongoing cost. Conversely, nothing in `dotnet-rs` breaks if either verifier repository goes quiet: the pinned Rust crates still expand and type-check, and the pinned manifest still describes exactly what was once proved.

7 VESPROOF: an embedded proof-script DSL

This section makes the recommendation of section 6.2 concrete enough to evaluate: a tactic-oriented proof-script language that lives *inside* the Rust source as a block-shaped macro adjacent to each unsafe site, its expansion into ordinary Rust (ghost tokens the compiler itself checks), its deterministic elaboration into Lean 4 statements, the diagnostics model, and three worked examples from the real corpus — including one where the proof now fails *at compile time*, as intended.

7.1 Design lineage: the `asm!` pattern from `dotnetdll`

The project already contains a stylistic precedent for embedded DSLs adjacent to Rust code. `dotnet-rs`'s metadata layer is built on the author's `dotnetdll` crate [9], whose `asm!` macro embeds a CIL assembler directly in Rust:

```
Some(body::Method::new(asm! {
    load_string "Hello from dotnetdll!";
    call write_line;           // `write_line` is an ordinary Rust binding
    Return;
}))
```

Four properties of this macro family transfer directly to proof scripts; VESPROOF adopts all four:

1. **Statement shape.** Each line is a keyword-led statement terminated by `;` — not an attribute's nested meta-item tree. Scripts of ten steps stay readable; attribute syntax at that size does not.
2. **Native splicing.** Operands are ordinary Rust expressions resolved in the enclosing scope (`call write_line`). No string quoting, no name mangling, ordinary hygiene.
3. **Binders into scope.** `asm!`'s `@label` form introduces Rust `let` bindings usable later in the block. Proof scripts need the same shape for premise witnesses: `have parked = ...;` binds a value later steps consume.
4. **One declarative source of truth.** `dotnetdll`'s entire `Instruction` enum — variants, flag fields, constructors, pretty-printers — is generated by a declarative macro from one compact specification. The VESPROOF vocabulary (goal and premise constructors) uses the same technique, generating the Rust token types *and* the Lean declarations from a single versioned source, so the two sides cannot drift (section 6.5).

The earlier draft of this study proposed a `#[ves::proof(goal, using, lemma)]` attribute. That form is kept only as a degenerate case (single-obligation sites with an already-proved lemma); the primary surface is the script block below, for the same reason `dotnetdll` did not encode CIL methods as attribute arguments.

7.2 Surface syntax

`ves::proof!` is an expression-position macro. Its body is a proof script; its final statement is the protected expression, which the macro re-emits *verbatim* — the block evaluates to whatever the protected expression evaluates to.

```
// SAFETY: prose stays; the script is the checked artifact.
let owner_id = ves::proof! {
  goal read_frozen(ObjectInner::owner_id) at lock_ptr;
  have parked    = stw::all_mutators_parked();
  have alive     = arena::pinned_while_parked(parked, lock_ptr);
  have wonce     = field::write_once(ObjectInner::owner_id);
  have visible   = stw::handshake_publishes(parked, wonce);
  show deref_frozen(alive, visible);
  then unsafe { (*lock_ptr).as_ptr().owner_id() }
};
```

The statement forms:

- `goal g at e1, ...`; — names the obligation, an instance of a vocabulary goal constructor applied to spliced Rust expressions and paths. *Optional*: for the common shapes the vocabulary carries an inference table keyed on the protected expression’s syntax (raw `deref` $\Rightarrow$ `read_valid` / `write_valid`; `from_raw_parts` $\Rightarrow$ `slice_valid`; `AtomicUN::from_ptr` $\Rightarrow$ `atomic_access`), so short scripts start at `have`. An explicit goal always overrides inference, and the checker records which of the two produced the obligation.
- `have x = t`; — acquires a premise witness. The right-hand side is a *token expression*: a call into the vocabulary’s ghost-token API (section 7.3), whose arguments may be earlier witnesses (the premise DAG is therefore type-checked by `rustc`) or spliced Rust values.
- `trust x = registry::name`; — acquires a trusted-premise token. Compiles only if `name` is declared in `TRUSTED.toml` (section 7.3); counted by the ratchet.
- `show tac(x1, ...)`; — the discharge step: a *tactic expression* naming a vocabulary tactic applied to witnesses. Tactics may nest (`show frame(alive, arith())`) and an escape hatch `show lean "..."` carries a raw Lean tactic block for sites that outgrow the surface vocabulary. `show` is a proof *sketch*, not the proof: it elaborates to the head of a Lean tactic script (section 7.5), and the site theorem in the `proofs` package may extend or replace it.
- `then e` — the protected expression or item, emitted verbatim. Exactly one per block, always last.
- `note "..."`; — attached prose, carried into the manifest for reviewers; no semantic content.

An item-position twin, `ves::proof_impl! { ... then unsafe impl ... }`, covers the 29 `unsafe impl` sites; a `ves::exempt! (reason = "...")` form covers deliberate opt-outs. The grammar is deliberately regular — every statement is keyword `...`; — so the parser is a few hundred lines of `syn` in the shared `ves-syntax` crate, and *the same parser* is linked into both the proc macro and the external checker, eliminating grammar skew by construction.

7.3 Premises as ghost tokens

The single largest improvement over the attribute draft is that premises are not strings the external checker inspects — they are zero-sized Rust values with private constructors, generated from the vocabulary source into the tiny runtime crate `ves-tokens`. A `have` statement expands to an ordinary (inlined, empty, release-free) function call returning a ZST:

```
// generated in ves-tokens (simplified)
pub struct Parked(()); // ZST, private ctor
pub struct PinnedWhileParked<'a>(PhantomData<&'a ()>);

pub mod stw {
    #[inline(always)]
    pub fn all_mutators_parked() -> Parked {
        debug_assert!(crate::hooks::is_stw_in_progress());
        Parked(())
    }
}
```

Token constructors come in four trust classes, mirroring section 3.2:

- *Static* — acquirable anywhere; may carry a `debug_assert!` shadow (as above) so the fixture suite dynamically spot-checks the premise for free.
- *Checked* — the constructor is defined *next to the runtime guard it names and under the same cfg*. This is the central rule: a guard that is compiled out cannot mint its token, so citing it is a `rustc` error in exactly the feature sets where the citation is false. Defect (a) of section 2.3 becomes unwritable (section 7.9).
- *Trusted* — the constructor’s argument is a `TrustGrant` `const` that a build script generates from `TRUSTED.toml`; an unregistered trusted premise does not compile, and the ratchet counts grants, not comments.
- *Derived* — constructors that consume other tokens (`pinned_while_parked(parked, ptr)`), so the dependency structure among premises is enforced by the type checker rather than by checker lint rules.

Two properties constrain this layer. *Zero cost*: every token is a ZST, every constructor is `#[inline(always)]` and empty in release; the pinned layout-size tests extend to `assert size_of` of every token is 0. *Token-identity*: the macro emits the `then-expression`’s tokens unchanged — `VesProof` never rewrites the `unsafe` code it annotates — and this is testable by expansion snapshot (`cargo expand diff` against the raw expression), which reduces the principal proc-macro risk to a CI check.

What remains for the *external* structural checker is what `rustc` cannot see: that every `unsafe` block in designated crates is wrapped at all, that obligation hashes match the proof manifest (section 7.5), staleness policy, and the trusted-premise budget. The external checker’s role narrows while the guarantees strengthen: name resolution, `cfg`-liveness, and premise dataflow all move into the ordinary Rust build, where they are checked in every feature configuration the CI matrix already covers.

7.4 Binding, hygiene, spans, and `cfg`

Binding by construction. Because the script *wraps* the protected expression rather than sitting near it, there is no adjacency convention to enforce and no way for refactors to separate proof from code: move the expression and the script moves with it; delete the expression and the script dies with it.

Hygiene. have-bound witnesses are ordinary `let` bindings introduced by the macro at the block’s scope (the `asm! @label` pattern); spliced expressions (`lock_ptr`, `self.layout`) resolve in the caller’s scope under standard `macro_rules/proc-macro` hygiene. The macro introduces no identifiers except the user’s binders, and all vocabulary paths are absolute (`::ves_tokens::stw::...`), so the expansion cannot capture or be captured.

Spans. The `proc` macro preserves the token spans of every script statement and of the protected expression, so (i) rustc errors inside the `then-expression` point at the user’s code exactly as if the macro were absent, and (ii) the obligation manifest records a real `file:line:col` span per statement, which the bridge uses to map Lean-side failures back to source (section 7.6).

cfg-awareness. The expansion is subject to the same `cfg` evaluation as the site itself, which resolves the question the attribute draft handled with a bespoke checker rule: a script under `#[cfg(feature = "multithreading")]` simply does not exist in single-threaded builds, and its obligations are extracted per active feature set. The extraction pass runs once per CI matrix leg (the matrix already exists), and the manifest keys obligations by `(site, cfg-set)` — the same site may legitimately carry different proofs under different features, which is the reality of this codebase (`ThreadSafeLock`’s two backends).

7.5 Deterministic elaboration to Lean 4

The pipeline from script to theorem has three mechanical stages and one human one:

1. **Extraction.** `ves-check extract` parses the crate with `ves-syntax` (the same parser the macro links) and emits the *obligation manifest*: for each site, the goal term (after inference), the premise list with trust classes, the tactic sketch, the span, the active `cfg` set, and two hashes — the *statement hash* over the canonicalized goal+premise terms, and the *subject hash* over the protected expression’s token stream. Canonicalization is total and documented (sorted premise sets, normalized paths, no source positions in the hash input), so extraction is deterministic across machines and macro-expansion order.
2. **Statement generation.** A Lean metaprogram in `VesCore` consumes the manifest and elaborates each goal into a `Prop` over the theory packages — the same elaboration rules are versioned with the vocabulary, and the Lean side recomputes the statement hash from its elaborated term as a cross-check. For new obligations it emits a stub:

```

/-- site dotnet-value/src/object/mod.rs:288 (cfg: multithreading) -/
theorem site_9f3ac1 :
  ReadFrozen ObjectInner.owner_id lockPtr := by
  ves_sketch [all_mutators_parked, pinned_while_parked,
              write_once, handshake_publishes, deref_frozen]
  sorry

```

3. **Proof.** A human or agent replaces `sorry`, using the sketch tactics — which are ordinary Lean tactic macros defined in the theory packages, i.e. *untrusted automation whose output the Lean kernel checks*, the `RefinedRust/Lithium` discipline [16]. Reusable surface tactics (`deref_frozen`, `frame`, `stw_open`) are exactly reusable Lean lemma-combinators; growing the tactic library is proof engineering, never kernel engineering. This is where the bespoke-kernel temptation of section 6.1

is permanently discharged: the surface DSL gets arbitrarily domain-fitted while the TCB stays at Lean’s kernel plus the elaboration rules.

4. **Publication.** DotnetRsProofs’ CI exports the *proof manifest*: obligation hash $\rightarrow$ theorem name, sorry-free flag, axioms reachable from the proof (`#print axioms transitively`), toolchain and vocabulary versions. `dotnet-rs`’s CI diffs its freshly extracted obligation manifest against this artifact — no Lean toolchain anywhere in the VM’s build (section 6.5).

Statement-hash discipline gives the drift semantics: change a script’s goal or premises and the hash moves — the site reverts to *unproved* (blocking or ratcheted, per policy) until the Lean side catches up; change only the protected expression and the subject hash moves — a weaker “re-audit” state, since the statement may still hold, flagged rather than blocked by default. Both states are visible in the checker’s report, which is the mechanized version of the re-audit triggers section 8.2 asks for.

7.6 Diagnostics

Three failure surfaces, three reporting paths, all span-carrying:

- *Script errors* (unknown vocabulary item, malformed statement, missing then) are proc-macro parse errors — ordinary rustc diagnostics at the offending token, with the vocabulary crate supplying “did you mean” candidates.
- *Premise errors* (token constructor cfg’d out, unregistered trust grant, dataflow type mismatch) are ordinary type errors at the have statement’s span — rustc’s own machinery, in every matrix leg.
- *Proof-state errors* (hash mismatch, sorry, staleness, budget breach) come from `ves-check`, which formats them rustc-style against the manifest’s recorded spans, including the Lean theorem name and the last-good hash so the fix is mechanical to locate.

7.7 Worked example 1: the cross-arena rebrand (F1+F6)

The principal accepted-risk boundary (`heap.rs:436--445`) in script form:

```
ves::proof! {  
  goal rebrand_confined(ptr => tracing_arena) in trace_call;  
  have parked = stw::all_mutators_parked();  
  have pinned = stw::arenas_pinned_while_parked(parked);  
  have fresh = arena::recorded_cross_ref(ptr, generation);  
  have scoped = scope::no_escape_of_result();  
  show rebrand_ok(pinned, fresh, scoped);  
  then unsafe { Gc::from_ptr(ptr.as_ptr()).trace(cc); }  
}
```

`scope::no_escape_of_result()` is the one premise discharged syntactically by the macro itself (the handle is consumed within the `then`-expression; no binder escapes), recorded in the manifest as macro-checked. The interesting premise is `arenas_pinned_while_parked` — a derived token whose Lean counterpart is *the* protocol theorem (section 8.4): in phase `parked`, no arena teardown transition exists and `unregister_arena` blocks on `active_leases`. Until that theorem lands, the site compiles with `trust pinned = registry::stw_pins_arenas`; instead — one changed line, one ratchet entry, and the flip from `trusted` to `proved` later is a one-line diff whose hash the checker verifies.

7.8 Worked example 2: tracing completeness (F5+F2)

The item-position form on FieldStorage’s Collect impl:

```
ves::proof_impl! {  
  goal trace_complete(FieldStorage) via gc_desc(layout);  
  have parked = stw::all_mutators_parked();  
  have visible = stw::handshake_publishes(parked,  
                                           field::all_of(FieldStorage));  
  trust faithful = registry::gc_desc_faithful;  
  show trace_via_desc(visible, faithful);  
  then unsafe impl<'gc> Collect<'gc> for FieldStorage { /* ... */ }  
}
```

Elaboration splits the goal into the per-type completeness statement (every reference the bytes contain sits at an offset GcDesc marks) and the data-race side condition (discharged by the same STW theorem as example 1 plus the handshake happens-before lemma, closing the gap the prose hand-waves in F7). `gc_desc_faithful` starts life as a trust grant and is retired in Phase 4 by the `GcDescFaithful` theorem against the mechanized §II.10.7 rules — provable once per layout algorithm, and testable against the executable model on the fixture corpus’s types *before* it is proved.

7.9 Worked example 3: a proof that fails at compile time (F4)

The defective site from section 2.3(a), scripted honestly:

```
let v = ves::proof! {  
  // goal inferred: atomic_access(u32) at ptr  
  have aligned = checked::alignment_guard(ptr, align_of::<u32>());  
  //      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ rustc E0425 in default  
  //      features: constructor exists only under  
  //      cfg(feature = "memory-validation"), same cfg as  
  //      the guard it witnesses.  
  show atomic_from_witness(aligned);  
  then unsafe { AtomicU32::from_ptr(ptr as *mut u32) }.load(ord)  
};
```

In the attribute draft this was caught by checker rule 3; here it is not writable at all — the token constructor is `cfg`-mirrored with `validate_alignment`, so the build that ships without the guard is the build in which this script does not compile. The author’s options are unchanged and all sound: acquire the witness from the always-on guard (`is_ptr_aligned_to_field` returns its token in every configuration), push a contract token onto the pub `unsafe fn` boundary so every caller supplies it, or route through the internally-checked `Atomic::load_field`. The earliest possible detection moved from CI to the editor.

7.10 Treatment of template comments

Unchanged in substance from the attribute draft, upgraded in mechanism: the 342 boilerplate comments collapse into contract tokens on the ~62 documented `unsafe fn` — the function’s `#Safety` section becomes a `ves::contract!` declaration minting a per-function contract token type; each caller’s script acquires it (`have c = contract::write_to_unmanaged(bounds);`) by supplying

the bounds-check witness; the callee’s own script consumes `c` as its ambient precondition. One contract, one lemma, dozens of sites, and the chain from caller check to callee deref is a typed dataflow `rustc` verifies — the “documented preconditions and preceding bounds/origin checks” template sentence becomes a value with a definition. The bimodality of the corpus (section 2.2) remains the reason the campaign is tractable: ~25 goal constructors, ~40–60 real lemmas, a handful of protocol theorems.

8 Risks and limitations

This section assesses the principal failure modes. Six are structural; none is fatal; two reorder the roadmap.

8.1 The trusted computing base, itemized

The design of section 7 moves almost everything out of the TCB; what remains warrants explicit enumeration. Trusted for *soundness of the theorems*: the Lean kernel; the goal elaborator in `VesCore` (vocabulary term $\rightarrow$ Lean Prop — kept small, data-driven from the vocabulary source, and cross-checked by recomputing statement hashes on both sides); and the axiom packages (`RustAssumptions`, the trust-grant registry, and the standard’s own undefined-behavior latitude). Explicitly *untrusted*: the proc macro, the surface tactics (Lean metaprograms whose output the kernel checks), all proof automation, and `ves-check` itself — a structural-rule engine whose failure modes are false alarms or missed bookkeeping, not false theorems.

Trusted for *correspondence* — the claim that the theorem is about the code in the repository — are two mechanically narrowed items. First, the extraction path: the macro and the checker link the same `ves-syntax` parser, so there is one grammar, and the subject hash covers the protected expression’s exact token stream. Second, the token-identity property: the macro must re-emit the then-expression verbatim, which is enforced by expansion snapshots in `ves-macros`’ own CI. What no hash can close is the semantic gap between the extracted goal term and the operational meaning of the surrounding Rust — that gap is section 8.2, and full closure means MIR-level extraction (`RefinedRust`’s move), out of scope for the first campaign and identified here as the principal residual weakness.

One genuinely new risk arrives with the ghost-token layer: the macro expands inside the crate under verification, so a bug in `ves-macros` could in principle alter behavior. The mitigations are the token-identity snapshot tests, the `ZST/inline(always)` discipline (extended into the existing pinned layout-size test matrices: every token asserts `size_of == 0`), and the fact that the expansion is deliberately minimal — `let` bindings of unit structs plus the verbatim subject. This risk class is shared with every proc macro the workspace already uses, including `dotnet-macros` and `dotnetd11`’s own [9].

8.2 The model-fidelity gap

Every theorem is about $\mathcal{M}_{\text{VES}}$, $\mathcal{M}_{\text{Rust}}$, and hand-stated correspondences — not about the compiled binary. The gap has three layers, with different mitigations:

- *VES model vs. ECMA-335 prose*: mitigated the JSCert way [7] — the model is executable and differentially tested against real .NET via the existing 486-fixture corpus and diff harness, which today has exactly one differential test and deserves expansion regardless of this project.
- *Rust axiomatization vs. rustc reality*: the $\mathcal{M}_{\text{Rust}}$ axioms are aligned with MiniRust’s UB list [26] and Tree Borrows [42], and every axiom gets a Miri-executable counterexample template: if the axiom is wrong in a way that matters, a fuzz/Miri run should be able to exhibit it. Residual risk is

real and stated: theorems are conditional on the axiomatization (~30 axioms is the target ceiling; RefinedRust-grade foundations remain the upgrade path for critical modules).

- *Script vs. code*: largely mechanized by the design of section 7.5. Statement drift (the goal or premises changed) moves the statement hash and demotes the site to unproved; subject drift (the protected expression changed under an unchanged statement) moves the subject hash and flags re-audit. The residue is drift the hashes cannot see — changes to *surrounding* code that invalidate a premise without touching the site (a new caller that skips a bounds check, a moved guard). The contract-token mechanism (section 7, templates) closes the caller case by typed dataflow; the rest is what the cross-repo drift job and the falsifier suites exist to catch.

8.3 Refactor churn under an LLM-agent workflow

This codebase refactors at a pace conventional verified projects never face — 44-step supervised campaigns are routine here. The embedded form is markedly better under churn than external annotations: scripts are ordinary Rust syntax, so `rustfmt` formats them, rename-refactors rewrite spliced identifiers along with everything else, moving a function moves its proof, and deleting code deletes its obligations rather than orphaning them. What churn still costs is proof-manifest staleness — statements whose hashes move faster than agents re-prove them. Three design choices exist specifically for this: obligations factored per *family* rather than per site (renames move instantiations, not theories); the statement/subject hash split (site motion re-anchors; only meaning changes re-prove); and the lag-tolerant manifest policy of section 6.5, so `cargo build` and the fast CI never wait on Lean. The realistic posture: proof repair becomes a normal part of the finalize step in the existing refactor convention, and agents do the first pass — the Lean ecosystem is currently the strongest available for that loop. The corollary is a scoping rule: *annotate the stable core (value representation, GC boundary, atomics), not the churning periphery (instruction handlers, intrinsics)*, at least until the model layer has demonstrated its value.

8.4 The stop-the-world protocol theorem

The single theorem most of the corpus leans on — all mutators parked $\Rightarrow$ foreign allocations live and stable — ranges over the coordinator’s lock-backed session model, the thread manager’s handshake, panic-unwind guards (`ResumeOnPanic`, `CommandCompletionGuard` typestate), safepoint-exclusion counters, and the lease/generation machinery of arena teardown. This is a genuine concurrent-protocol verification, the kind of thing rely-guarantee and Iris ghost state were built for [45, 25]. It is weeks-to-months of work for one person even with the LTS carefully cut down, and there is no shortcut: without it, the F1 family rests on a trust grant. The mitigation is sequencing — the protocol model (not proof) comes early, gets model-checked exhaustively for small configurations (2–3 threads) to expose specification errors cheaply, and only then graduates to a full proof. The existing lock-order DAG and the documented forbidden inversions are a head start: they are, in effect, hand-written lemmas about the LTS.

8.5 Trusted-premise creep

The two irreducible trust classes (section 3.2) — spec-licensed UB from unverifiable IL, and FFI ABI agreement — are legitimately axiomatic. The failure mode is sociological: `trust` is the path of least resistance, and a registry that grows monotonically converts the proof layer back into prose with extra steps. The `TrustGrant` mechanism helps — an unregistered trusted premise does not compile, so the

registry cannot silently lag the code — but the ratchet still requires enforcement: CI reports the grant count as a budget exactly like the existing `check_mt_cfg_ceiling.sh`, and each grant must name the cheapest known falsifier (a Miri mode, a fuzz target, a fixture) so trust is at least monitored. Two specific entries deserve flagging now: the *non-moving GC* axiom (fine today, invalidated the day a compacting collector lands — the grant should say so) and the `TypeIndex` transmute into a foreign crate’s private layout, which is better fixed upstream in `dotnetdll` (a `#[repr(transparent)]` guarantee or a constructor) than trusted here — the author owns that crate too.

8.6 Provenance

By design, the managed-pointer serialization calls `expose_provenance` in its 3-word encoding and so launders provenance through `usize`; strict-provenance Miri is documented infeasible, and no tool in CI checks provenance obligations today. The Lean-side $\mathcal{M}_{\text{Rust}}$ can and should model exposed provenance (the angelic-choice semantics MiniRust specifies), which makes the obligations *statable*; but the axioms here are the least falsifiable of the set, and the study flags pointer serialization/deserialization as the module where proof adds the least confidence per unit effort until upstream tooling improves.

8.7 Cost model

Order-of-magnitude estimates for a solo developer with strong type theory, working with LLM agents, part-time on this track (calibrated against: RefinedRust team-years for foundational unsafe proofs; Hawblitzel-Petrack two-person SMT-automated GC verification [19]; JSCert/Jinja multi-year mechanizations; and the observed productivity of agent-assisted Lean):

Artifact	Effort	Risk	Standalone value
CI-gate hardening + defect fixes (a),(b)	days	nil	high, immediate
ves-syntax + vocabulary v0 + codegen	2–3 weeks	low	the anchor
ves-macros + ves-tokens + snapshots	2–4 weeks	low	rustc-checked premises
ves-check + manifests + ratchet	1–2 weeks	low	tag-std-class tool
$\mathcal{M}_{\text{VES}}$ core, executable + diff-tested	3–6 months	medium	high (first of its kind)
STW/arena protocol LTS + model checking	1–2 months	medium	high (spec-bug hunting)
$\mathcal{M}_{\text{Rust}}$ axiomatization	1–2 months	medium	moderate
Protocol theorem (F1 spine)	1–3 months	high	the keystone
Layout faithfulness (F2) + GcDesc theorem	1–2 months	medium	high
Critical-site campaign (~40–60 lemmas)	3–6 months	medium	the stated goal

Summed naively: 1–1.5 years of part-time focused work to the full stated goal, with usable, independently valuable artifacts landing from week one. The embedded-DSL front end adds roughly two to four weeks over the attribute draft and buys rustc-checked premise discipline in every build; the estimate’s dominant uncertainty remains the protocol theorem and the degree to which agents can be trusted with Iris-style ghost reasoning versus plain LTS invariants. The roadmap (section 9) is cut so that a stall there strands nothing else.

9 Roadmap

The package decomposition of section 6.5 fixes the shape; what remains is the order. The sibling `ves-proof` repository hosts the five Rust crates under `crates/`; `ves-proof-lean` hosts the five independent Lake packages under `packages/`. Their base skeletons now exist. `dotnet-rs` gains scripts, two

dev-dependencies, one CI job, and `TRUSTED.toml`. Phases, each independently valuable and abandonable without stranding the previous:

Phase 0 — Hygiene (days). Fix the two live defects (section 2.3): real alignment guards (or caller contracts) on `StackValue::load_atomic/store_atomic`, and either a checked discipline or a narrower structure for the `SharedGlobalState` auto-trait overrides. Decide which dynamic gates can be promoted from `continue-on-error` to blocking (even a single Miri leg on `dotnet-value` would be the first *enforced* UB gate). None of this needs the DSL; all of it raises the floor the DSL builds on.

Phase 1 — The embedded layer (4–7 weeks). In order: in `ves-proof/crates/`, `ves-syntax` and the declarative vocabulary `v0` (the F1–F9 constructors and their trust classes) with its two code generators; `ves-tokens` with `cfg-mirrored` constructors and `TrustGrants` generated from `TRUSTED.toml` (seeded from the accepted-risk boundaries); `ves-macros` with token-identity snapshot tests; `ves-check` in structural mode (empty proof manifest, everything trusted or unproved); scripts on every unsafe block in `dotnet-value`, `dotnet-utils`, and `dotnet-runtime-memory`, and `ves::contract!` declarations on the 62 documented unsafe fn. Deliverable: premise existence, `cfg-liveness`, and premise dataflow checked by `rustc` in every CI matrix leg — defect (a) becomes uncomparable (section 7.9) — plus structural coverage enforcement. This is a state-of-the-art contribution at (beyond) the tag-std level [1] before any theorem exists, and it is also the test of the central hypothesis: if scripts cannot survive this project’s refactor velocity, the lesson costs a linter rather than a model.

Phase 2 — The model (3–6 months). `VesModel`: in `ves-proof-lean/packages/VesModel/`, $\mathcal{M}_{\text{VES}}$ in Lean 4 — layout rules, pointer taxonomy, memory/atomicity model, eval-stack typing, reachability/finalization; executable; differentially tested against real `.NET` on the fixture corpus (which also expands the diff harness the project needs anyway). Independent of Phase 1’s crates by design. Deliverable: the first mechanized ECMA-335 core — publishable, and useful as a specification oracle even if no later phase completes.

Phase 3 — The two hard theories (2–4 months, parallelizable). Under `ves-proof-lean/packages/`, `VesProtocol`: the STW/arena LTS, model-checked exhaustively at small configurations, then its invariants proved. `RustAssumptions`: the $\mathcal{M}_{\text{Rust}}$ axiomatization with Miri-falsifier templates. `VesCore`’s goal elaborator and stub generator land here, making the manifests flow end-to-end. Gate: the protocol theorem (section 8.4) — if it stalls, F1 sites stay on trust grants and the campaign continues at reduced strength.

Phase 4 — The campaign (3–6 months, then open-ended). In `ves-proof-lean/packages/DotnetRsProofs/`: the representation relation $\mathcal{R}$; `GcDescFaithful`; then the critical sites in value order — the two accepted-risk boundaries, the `Collect` impls, the atomics family, the `P/Invoke` pinning contracts. Each proved lemma retires a trust grant or an unproved hash under the ratchet, a one-line script diff per site (section 7.7). Stretch goals beyond the stated scope, in order of leverage: MIR-level tethering for the critical modules; a mechanized CIL verifier fragment to discharge the managed-IL trust class; `RefinedRust/Rocq` treatment of the `gc-arena` boundary.

Two standing rules constrain the effort: every phase must deliver an artifact `dotnet-rs` uses in CI within that phase, and the trusted-premise ratchet only moves downward.

10 Decomposition into supervised agent refactors

The roadmap of section 9 fixes phase order; this section maps the phases onto the project’s existing execution machinery. `dotnet-rs` is developed through supervised, agent-executed refactor sessions managed by a local orchestrator (`nasin-pali-pi-jan-ilo`, state in `.refactor-orchestrator/`): each session binds one repository and one branch to one written goal, a kickoff agent produces a review document and a checklist of anchored steps (`- [ ] N.M ... --- refs REVIEW.md#anchor`), each step is executed by a fresh-context agent at a configured effort tier (`draft/standard/deep/max`) and must attach machine-checkable verification evidence, and the supervisor reviews the per-step diff before the step commits. The repository’s history contains 51 such sessions, all merged, ranging from 6 to 68 checklist steps. The decomposition below is stated in that system’s units.

10.1 Sizing measures

Wall-clock estimates are not meaningful for agent-executed work; the quantities that actually predict whether a session succeeds are:

- **Checklist size class.** S (≤ 10 steps), M (11–25), L (26–45). Prior sessions above ~ 45 steps correlate with plan defects (mass *blocked* reports mid-session), so any package estimated at L or larger below is pre-split.
- **Verification-signal quality** (*high / medium / low*): whether each step has a machine-checkable acceptance test an agent can iterate against without supervision. Lean packages are the extreme high case — a sorry-free lake build is a sound per-step gate, so `draft`-tier retry loops are usable for proof search. Design-type steps (choosing a Prop’s statement, cutting an LTS) are the low case: the artifact compiles regardless of whether it says the right thing, so tier and supervisor attention concentrate there.
- **Step context-independence:** whether a fresh-context agent can execute a step from the checklist line, the review anchor, and `AGENT_MEMORY.md` alone. Grammar-and-codegen work is highly independent; annotation sweeps are independent per file; model design is not, and such sessions need richer review documents at kickoff.
- **Supervisor gate density:** the fraction of steps whose diff requires a substantive human judgment rather than a conformance glance. This is the binding resource. It is highest where statements (not proofs) are produced: vocabulary elaboration rules, axioms, LTS transitions, trust grants.
- **Blast radius:** single-crate sessions merge cleanly; sessions that touch shared files across crates serialize. Prior experience with cascade refactors indicates slicing by crate dependency order, not by semantic theme.

Parallelism operates at the session level, not the step level: a session owns one branch of one repository, so *sessions in different repositories run concurrently*, while sessions in the same repository serialize (annotation sweeps within `dotnet-rs` could in principle run on disjoint crates, but shared-file conflicts and the one-branch convention argue against it). The package structure of section 6.5 was chosen partly for this reason: after WP-2, the three lanes below are independent repositories.

10.2 Ordered work packages

Each entry is one orchestrator session (or an explicitly noted short series), with a goal fragment of the kind the kickoff form accepts. Lanes: **A** = dotnet-rs, **B** = ves-proof/crates/, **C** = ves-proof-lean/packages/. Within a lane, order is strict; across lanes, an entry lists its cross-lane prerequisites.

1. **[A] Atomic-path soundness.** Close the unguarded `StackValue::load_atomic` and `store_atomic` paths and restructure the `SharedGlobalState` auto-trait overrides behind a checked discipline. (*S; deep; high signal — Miri leg + existing atomic tests.*)
2. **[A] Dynamic-gate promotion.** Promote the stable Miri and fuzz legs from `continue-on-error` to blocking and extend the differential harness beyond its single fixture. (*S; standard; high signal — CI is its own oracle.*)
3. **[B/C] Repository seeds (scaffolded).** Initialize the ves-proof Rust workspace and its five crate skeletons under `crates/`; initialize the five independent Lake package skeletons under `ves-proof-lean/packages/`; reserve the two manifest schemas in `ves-proof/schemas/`. The base skeletons exist; CI and implementation work remain governed by the later packages. (*S; standard; done once, largely mechanical.*)
4. **[B] ves-syntax.** Implement the script grammar, AST, canonical serializer, and statement/subject hashing against a golden corpus of scripts. (*M; standard; high signal — golden files and round-trip properties make every step self-checking.*)
5. **[B] ves-vocabulary.** Define the declarative vocabulary source for the F1–F9 constructor families and build the dual code generators (Rust token types, Lean declarations). (*M; deep for the schema steps, standard for codegen; the elaboration-rule steps carry the highest gate density in lane B.*)
6. **[B] ves-tokens.** Generate the ZST token crate: trust-class constructor forms, `cfg-mirroring` conventions, the `TrustGrant` build script, and zero-size/`inline` test matrices. (*S–M; standard; high signal.*)
7. **[B] ves-macros.** Implement `proof!`, `proof_impl!`, and `contract!` over `ves-syntax`, with expansion-snapshot tests establishing the token-identity property. (*M; deep — proc-macro hygiene and span work; snapshot tests give high signal once the harness exists.*)
8. **[B] ves-check.** Implement extraction, the structural rules, manifest comparison, and the ratchet budgets; fixture-test against synthetic crates. (*M; standard; high signal.*)
9. **[A] Integration bootstrap.** (Requires 6–8.) Add the `ves-macros/ves-tokens` dependencies, the `xtask ves-check` CI job in structural mode, and `TRUSTED.toml` seeded from the accepted-risk boundaries. (*S; standard.*)
10. **[A] Contract layer.** Declare `ves::contract!` on the ~62 documented `unsafe fn` and collapse the template comments onto contract tokens. (*M; standard; per-function steps are fully context-independent.*)
11. **[A] Annotation sweep, in dependency order.** Convert `// SAFETY:` sites to scripts one crate per session: `dotnet-utils` (M), then `dotnet-value` (L, *pre-split* into pointer/object/storage and stack-value/layout halves), then `dotnet-runtime-memory` (M). Per-site steps are independent and mostly

- standard; sites in the invariant families F1/F5/F6 warrant deep. (*4 sessions; high signal — rustc token checking plus ves-check run per step.*)
12. [C] **VesCore statement layer.** (Requires 5.) Implement the vocabulary mirror, the goal elaborator, statement-hash recomputation, and manifest ingestion, cross-checked against ves-syntax hashes on a shared corpus. (*M; deep for elaboration rules — the highest gate-density work in the project, since these rules are TCB; standard elsewhere.*)
 13. [C] $\mathcal{M}_{\text{VES}}$ **core.** A series of four sessions in fixed order: (a) memory store, alignment, and atomicity (§I.12.6); (b) type layout including §II.10.7 and the reference-overlap rule; (c) pointer taxonomy and evaluation-stack typing (§II.14.4, §III.1.1.5, after Gordon-Syme); (d) reachability, finalization, and GC-handle semantics (§I.8.9.6.7). (*$4 \times M$; deep for definitional steps, draft-standard for lemma backfill; medium signal until 14 exists — definitions compile whether or not they are faithful, so supervisor review concentrates on statements.*)
 14. [C] **Differential harness.** Execute $\mathcal{M}_{\text{VES}}$ against the fixture corpus and real .NET; drive the model's fidelity backlog from divergences. (*M; standard; converts lane C to high signal retroactively — divergence lists become mechanical checklist items.*)
 15. [C] **VesProtocol model.** Define the STW/arena LTS and model-check it exhaustively at 2–3 threads; every counterexample becomes a checklist item against the model or, if real, an issue against dotnet-rs. (*M; deep for the LTS cut, standard for the checking harness; the LTS transitions are supervisor-heavy.*)
 16. [C] **RustAssumptions.** State the $\mathcal{M}_{\text{Rust}}$ axioms with one Miri-falsifier template per axiom. (*M; deep; low proof content but every statement is TCB — gate density comparable to 12.*)
 17. [C] **Manifest round trip.** (Requires 8, 12.) Land the stub generator and proof-manifest export, and run the first end-to-end obligation→stub→proof→manifest→ves-check cycle on a toy site. (*S; standard; high signal — the cycle either closes or does not.*)
 18. [C] **Protocol theorem.** (Requires 15.) Prove the pinning invariants of the LTS — the F1 keystone. (*M–L; max with draft-tier retry loops for subgoals; perfect signal, hardest content; the roadmap's designated stall point, isolated so a stall strands nothing.*)
 19. [C] **Representation relation and GcDescFaithful.** (Requires 13b.) Define $\mathcal{R}$ and prove layout faithfulness against the mechanized §II.10.7 rules, testing the shared corpus against the executable model first. (*2 sessions, M each; deep; medium-to-high signal.*)
 20. [C] **Site campaign.** (Requires 17; 18 for F1 sites, 19 for F2 sites.) Retire trust grants family by family — the two accepted-risk boundaries, then Collect completeness, the atomics family, and P/Invoke pinning — one session per family, each step one lemma with the sorry-free gate. (*$4\text{--}5 \times M$; mixed tiers; perfect signal; steps fully independent, and by this point the sessions are the routine kind the orchestrator was built for.*)
 21. [A] **Ratchet closure.** Flip ves-check from warning to blocking for the proved families and record the residual grant budget. (*S; standard.*)

10.3 Pipeline shape

The dependency structure yields three long-lived concurrent lanes with four synchronization points: lane B is unblocked immediately after the repository seeds (3) and runs 4→8 sequentially; lane A resumes at 9 once lane B delivers the macros; lane C starts at 12 as soon as the vocabulary (5) stabilizes and is otherwise independent of lane A entirely; and the only cross-lane joins are 9 ($A \leftarrow B$), 12 ($C \leftarrow B$), 17 ($C \leftarrow B$), and 20 ($C \leftarrow A$'s annotation sweep, which supplies the real obligation manifest). Within lane C, sessions 13a–d are sequential but 14 can begin once 13a lands, and 15/16 are mutually independent. At peak (after 9), three sessions run concurrently — one per repository — which matches the supervision model: gate density, not agent throughput, is the binding constraint, and the lanes are deliberately mixed so that at most one supervisor-heavy session (12, 15, 16, 18) is active at a time while high-signal sweeps (11, 13-backfill, 20) fill the remaining lanes.

11 Recommendation

Proceed — in the reshaped form this study argues for, and in the phased order that front-loads standalone value.

Adopt. An embedded, tactic-oriented proof-script DSL in the lineage of `dotnetd11's asm!` [9]: block-shaped `ves : : proof!` scripts wrapping each `unsafe` site, premises acquired as zero-sized ghost tokens whose existence, `cfg`-liveness, and dataflow `rustc` itself checks, goals drawn from (or inferred against) a single declarative vocabulary that generates both the Rust tokens and the Lean declarations. Scripts elaborate deterministically to Lean 4 statements; proofs are ordinary Lean terms driven by reusable domain tactics that are untrusted metaprograms; the kernel is Lean's; Curry–Howard is inherited, not reimplemented. The whole verifier lives in a separately versioned repository whose only contact with `dotnet-rs` is two tiny Rust dependencies and two hashed manifest artifacts (section 6.5) — the VM's build never touches Lean. Surface conventions stay aligned with RFC 3842 safety tags [46] so the project tracks, and can eventually contribute to, the community convention.

Reject. A bespoke proof kernel and solver (section 6.1: unauditable TCB, automation cost dwarfs the kernel, forfeits the agent ecosystem) — including its subtle reappearance as a “smart” checker that evaluates tactics outside Lean; wholesale adoption of any existing verifier as the whole answer (section 6.3: none reaches the ECMA-335 layer); attribute or comment syntax as the primary carrier for multi-step proofs (kept only as the degenerate single-lemma form); and any plan whose first deliverable is a proof rather than the token layer and the model (section 8.1: untethered proofs decay into documentation).

Immediate actions, independent of any spin-off decision. Phase 0 is due diligence, not verification: fix the two live defects, promote at least one Miri leg to blocking, and expand the differential harness past its single test. Phase 1 (the embedded layer) is the cheapest experiment that tests the whole thesis on real ground — and, uniquely among the phases, it yields value even with no Lean involvement: ghost tokens make premise citations compile-checked in every feature configuration, which would already have made defect (a) unwritable.

Two genuinely open decisions. First, how far to trust the $\mathcal{M}_{\text{Rust}}$ axiomatization before reaching for Rocq foundations — this study says: cap it at ~30 Miri-falsifiable axioms and revisit after the critical sites are proved. Second, whether the VES model targets the standard's semantics or `dotnet-rs's`

documented deviations (single load context, non-moving GC, STW-only collection) — this study says: model the standard, and register each deviation as a named trust grant, because the deviations are exactly the things most likely to change and the grant registry is what makes changing them safe.

The strongest reason to proceed is internal to the project. `dotnet-rs` has pushed Rust-native assurance — branded lifetimes, capability tokens, compile-time lock DAGs, validation magics, a written proof charter — about as far as it goes, and the residue is a well-defined, finite set of obligations that are theorems or they are nothing. The project’s own documentation already writes them as theorems in prose (“STW keeps every arena alive and every mutator stopped through this read”); its own meta-data layer already demonstrates the embedding idiom the proof language should use. The proposal, correctly scoped, is not adding a formal layer to an informal project; it is giving a project that already reasons formally a checker for the reasoning it already does — and producing, along the way, the first mechanized model of the specification it implements.

References

- [1] Artisan-Lab, Fudan University. *Annotating and Auditing the Safety Properties of Unsafe Rust*. arXiv:2504.21312; safety-tool applied to libstd covers 96.1% of public unsafe APIs. 2025. URL: <https://github.com/Artisan-Lab/tag-std>.
- [2] AWS and Rust Foundation. *Verify Rust Std: Lessons Learned from Verifying the Rust Standard Library*. arXiv:2510.01072; contract attributes upstreamed to nightly std. 2025. URL: <https://github.com/model-checking/verify-rust-std>.
- [3] AWS Kani team. *Kani: A Model Checker for Rust*. arXiv:2607.01504; CBMC-based bounded model checking. 2026. URL: <https://github.com/model-checking/kani>.
- [4] Sacha-Élie Ayoun, Xavier Denis, Petar Maksimović, and Philippa Gardner. “A Hybrid Approach to Semi-Automated Rust Verification”. In: *Proceedings of the 46th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2025. DOI: 10.1145/3729289. URL: <https://gillianplatform.github.io/publications/rust.html>.
- [5] Patrick Baudin et al. *ACSL: ANSI/ISO C Specification Language*. Frama-C project, CEA-List and Inria. 2026. URL: <https://frama-c.com/html/acsl.html>.
- [6] Nick Benton. “A Typed, Compositional Logic for a Stack-Based Abstract Machine”. In: *Asian Symposium on Programming Languages and Systems (APLAS)*. 2005. DOI: 10.1007/11575467_24.
- [7] Martin Bodin et al. “A Trusted Mechanised JavaScript Specification”. In: *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2014. DOI: 10.1145/2535838.2535876.
- [8] Egon Börger, Nicu G. Fruja, Vincenzo Gervasi, and Robert F. Stärk. “A High-Level Modular Definition of the Semantics of C#”. In: *Theoretical Computer Science* 336.2–3 (2005), pp. 235–284. DOI: 10.1016/j.tcs.2004.11.008.
- [9] Nick Clifford. *dotnetdll: A Rust Library for Reading and Writing .NET Metadata*. Version 0.3.0; includes the `asm!msig!type_ref!` embedded-DSL macro family. 2026. URL: <https://github.com/nickbclifford/dotnetdll>.
- [10] David R. Cok. *OpenJML: Deductive Verification of Java Programs with JML Annotations*. 2025. URL: <https://www.openjml.org/>.

- [11] Cryspen. *hax: Translating Rust into F* and Rocq for Verification*. v0.1.0; targets a panic-free safe-Rust subset. 2025. URL: <https://github.com/cryspen/hax>.
- [12] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. “Creusot: A Foundry for the Deductive Verification of Rust Programs”. In: *International Conference on Formal Engineering Methods (ICFEM)*. 2022. doi: 10.1007/978-3-031-17244-1_6. URL: <https://github.com/creusot-rs/creusot>.
- [13] Ecma International. *ECMA-335: Common Language Infrastructure (CLI)*. 2012. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-335/>.
- [14] Formal Land. *rocq-of-rust: Formal Verification of Rust Programs in Rocq*. 2026. URL: <https://github.com/formal-land/rocq-of-rust>.
- [15] Nicu G. Fruja. “Type Safety of C# and .NET CLR”. Abstract State Machines formalization; not mechanized. PhD thesis. ETH Zürich, 2006.
- [16] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. “RefinedRust: A Type System for High-Assurance Verification of Rust Programs”. In: *Proceedings of the ACM on Programming Languages* 8.PLDI (2024). doi: 10.1145/3656422. URL: <https://plv.mpi-sws.org/refinedrust/>.
- [17] Andrew D. Gordon and Don Syme. “Typing a Multi-Language Intermediate Code”. In: *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2001. doi: 10.1145/360204.360228.
- [18] Travis Hance et al. “VerusBelt: Semantic Soundness for Verus”. In: *Proceedings of the ACM on Programming Languages* 10.PLDI (2026). doi: 10.1145/3808325. URL: <https://iris-project.org/pdfs/2026-pldi-verusbelt.pdf>.
- [19] Chris Hawblitzel and Erez Petrank. “Automated Verification of Practical Garbage Collectors”. In: *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2009. doi: 10.1145/1480881.1480935. URL: <https://lmcs.episciences.org/1039>.
- [20] Son Ho and Jonathan Protzenko. “Aeneas: Rust Verification by Functional Translation”. In: *Proceedings of the ACM on Programming Languages* 6.ICFP (2022). doi: 10.1145/3547647. URL: <https://github.com/AeneasVerif/aeneas>.
- [21] Bart Jacobs et al. *Foundational VeriFast: Pragmatic Certification of Verification Tool Results through Hinted Mirroring*. arXiv:2601.13727. 2026. URL: <https://arxiv.org/abs/2601.13727>.
- [22] Bart Jacobs et al. *Towards Verifying Unsafe Rust Programs Against Rust’s Pointer-Aliasing Restrictions*. arXiv:2603.28326; adds Tree Borrows obligations to VeriFast. 2026. URL: <https://verifast.github.io/verifast/rust-reference/>.
- [23] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. “Stacked Borrows: An Aliasing Model for Rust”. In: *Proceedings of the ACM on Programming Languages* 4.POPL (2020). doi: 10.1145/3371109. URL: <https://plv.mpi-sws.org/rustbelt/stacked-borrows/>.
- [24] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018). doi: 10.1145/3158154. URL: <https://plv.mpi-sws.org/rustbelt/pop118/>.

- [25] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. “Iris from the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic”. In: *Journal of Functional Programming* 28 (2018). DOI: 10 . 1017 / S0956796818000151. URL: <https://iris-project.org/>.
- [26] Ralf Jung and the MiniRust contributors. *MiniRust: A Precise Specification of Rust’s Runtime Semantics*. Leading candidate for Rust’s normative operational semantics. 2026. URL: <https://github.com/minirust/minirust>.
- [27] Andrew Kennedy and Don Syme. “Design and Implementation of Generics for the .NET Common Language Runtime”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2001. DOI: 10 . 1145/378795 . 378797.
- [28] Gerwin Klein and Tobias Nipkow. “A Machine-Checked Model for a Java-Like Language, Virtual Machine, and Compiler”. In: *ACM Transactions on Programming Languages and Systems* 28.4 (2006). DOI: 10 . 1145/1146809 . 1146811.
- [29] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. “CakeML: A Verified Implementation of ML”. In: *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2014. DOI: 10 . 1145/2535838 . 2535841.
- [30] Andrea Lattuada et al. “Verus: A Practical Foundation for Systems Verification”. In: *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*. Distinguished Artifact Award. 2024. DOI: 10 . 1145/3694715 . 3695952. URL: <https://github.com/verus-lang/verus>.
- [31] Andrea Lattuada et al. “Verus: Verifying Rust Programs Using Linear Ghost Types”. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023). DOI: 10 . 1145/3586037.
- [32] Lean community. *iris-lean: A Port of the Iris Separation Logic to Lean 4*. MoSeL, UPred, IProp, invariants ported; no mechanized Rust semantics. 2026. URL: <https://github.com/leanprover-community/iris-lean>.
- [33] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. “Flux: Liquid Types for Rust”. In: *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2023. DOI: 10 . 1145/3591283. URL: <https://github.com/flux-rs/flux>.
- [34] Xavier Leroy. “Formal Verification of a Realistic Compiler”. In: *Communications of the ACM* 52.7 (2009). DOI: 10 . 1145/1538788 . 1538814.
- [35] Andreas Lochbihler. “Mechanising a Type-Safe Model of Multithreaded Java with a Verified Compiler”. In: *Journal of Automated Reasoning* 61 (2018). DOI: 10 . 1007/s10817-018-9452-x. URL: <https://www.isa-afp.org/entries/JinjaThreads.html>.
- [36] Niko Matsakis and the Types Team. *a-mir-formality: A Formal Model of the Rust Type System*. 2026. URL: <https://rust-lang.github.io/rust-project-goals/2025h2/a-mir-formality.html>.
- [37] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *International Conference on Automated Deduction (CADE)*. 2021. DOI: 10 . 1007/978-3-030-79876-5_37. URL: <https://lean-lang.org/>.
- [38] Nima Rahimi Foroushaani and Bart Jacobs. *Modular Formal Verification of Rust Programs with Unsafe Blocks*. arXiv:2212.12976. 2022. URL: <https://arxiv.org/abs/2212.12976>.
- [39] Rust Project and Ferrous Systems. *Adopting the Ferrocene Language Specification*. 2025. URL: <https://blog.rust-lang.org/2025/03/26/adopting-the-fls/>.

- [40] Adam Sandberg Ericsson, Magnus O. Myreen, and Johannes Åman Pohjola. “A Verified Generational Garbage Collector for CakeML”. In: *Journal of Automated Reasoning* 63 (2019). DOI: 10.1007/s10817-018-9487-z. URL: <https://cakeml.org/itp17.pdf>.
- [41] The Rocq Development Team. *The Rocq Prover (formerly Coq), Version 9*. Renamed at Rocq Prover 9.0, released 2025-03-12. 2025. URL: <https://rocq-prover.org/releases/9.0.0>.
- [42] Neven Villani, Johannes Hostert, Derek Dreyer, and Ralf Jung. “Tree Borrows”. In: *Proceedings of the 46th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Distinguished Paper Award; formalized in Rocq/Simuliris; implemented in Miri. 2025. DOI: 10.1145/3735592. URL: <https://iris-project.org/pdfs/2025-pldi-treeborrows.pdf>.
- [43] Jean Yang and Chris Hawblitzel. “Safe to the Last Instruction: Automated Verification of a Type-Safe Operating System”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2010. DOI: 10.1145/1806596.1806610.
- [44] Dachuan Yu, Andrew Kennedy, and Don Syme. “Formalization of Generics for the .NET Common Language Runtime”. In: *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2004. DOI: 10.1145/964001.964012.
- [45] Yannick Zakowski et al. “Verifying a Concurrent Garbage Collector with a Rely-Guarantee Methodology”. In: *Journal of Automated Reasoning* 61 (2018). DOI: 10.1007/s10817-018-9489-x.
- [46] zjp-CN. *RFC 3842: Safety Tags*. Opened 2025-07-31; open as of mid-2026. 2025. URL: <https://github.com/rust-lang/rfcs/pull/3842>.